



Safety-constrained Deep Reinforcement Learning control for human–robot collaboration in construction

Kangkang Duan ^a, Zhengbo Zou ^{b,*}

^a The University of British Columbia, Department of Civil Engineering, Vancouver, Canada

^b Columbia University, Department of Civil Engineering and Engineering Mechanics, NY, USA

ARTICLE INFO

Dataset link: <https://kangk906.github.io/safetycon.github.io/>

Keywords:

Deep reinforcement learning
Construction robots
Construction safety
Human–robot collaboration
Reference governor
Control theory

ABSTRACT

Worker safety has become an increasing concern in human–robot collaboration (HRC) due to potential hazards and risks introduced by robots. Deep Reinforcement Learning (DRL) has demonstrated to be efficient in training robots to acquire complex construction skills. However, neural network policies for collision avoidance lack theoretical safety guarantees and face challenges with out-of-distribution scenarios. This paper proposes a biomimetic safety-constrained DRL control system, inspired by vertebrate decision-making systems. A neural network policy serves as the "brain" for complex decision-making, while a reference governor layer functions like the spinal cord, enabling rapid responses to environmental stimuli and prioritizing safety. Theoretical safety guarantees related to robot dynamics including torque, joint angle, velocity, and distance were analyzed. Experimental results demonstrate that the proposed method achieves a 0% collision rate, providing a safe HRC mode in both static and dynamic construction scenarios.

1. Introduction

Construction is characterized by heavy-duty machinery and unstructured environments, casting significant and complex safety risks. These risks have been manifested in high-frequency incidents. According to [1], safety issues in construction are alarmingly frequent, and accident rates show an upward trend. Moreover, construction safety incidents have a significant financial impact, leading to increased project costs due to both direct and indirect losses. Estimates indicate these increased costs range from 3~6% of the total project cost in the UK, 7.9% in the US, and exceed 10% in China [2]. Human–robot collaboration (HRC) in construction leverages the strengths of automation and the irreplaceable cognitive capabilities of humans. While HRC has the potential to boost the efficiency of construction and relieve human workers from repetitive and physically-demanding tasks [3], allowing robots to occupy a shared workspace with human workers brings unique safety challenges [4].

The primary concern among workers in construction HRC is the physical risk associated with interacting with robots [5]. Conventional HRC applications employ physical safeguards (e.g., fences) to keep workers separated from robots for safety. In these setups, robots will immediately stop working if a worker crosses the designated safeguarded space [6], which potentially limits the efficiency of HRC. This is because HRC can be designed to fully consider safe conditions without the need for physical separation [7]. Compared to other sectors,

construction still lags in efficiency, and this kind of safety mode limits the productivity of construction sector [8]. Over the past two decades, the average annual productivity growth in construction has been 1%, compared to 2.8% for the global economy and 3.6% for manufacturing [9]. In certain manufacturing applications [10], rather than simply stopping the robot when a worker enters the robot's working area, the robot's motion is halted only when the distance between the worker and the robot falls below the minimum safety distance. The distance is determined based on the relative position, speed, and movement directions. Indeed, international standards on cobots such as ISO/TS 15066 [11] define the speed and separation monitoring (SSM) mode in HRC, where the robot is required to keep a safe distance according to its torque, velocity, and relative position.

Despite efforts being made in manufacturing, similar standards have not been proposed in construction to specify safety rules on construction sites. However, on-site construction often involves situations where workers must share the same workspace with robots. For example, workers are required to assist with crane operations for loading and unloading during material transportation. Although there are safety regulations in place, studies [12] have shown that 46% of struck-by accidents involve bystanders. These bystanders include workers assisting with crane activities or those performing other construction tasks nearby. Assembly and welding robots for complex components

* Corresponding author.

E-mail address: zz3279@columbia.edu (Z. Zou).

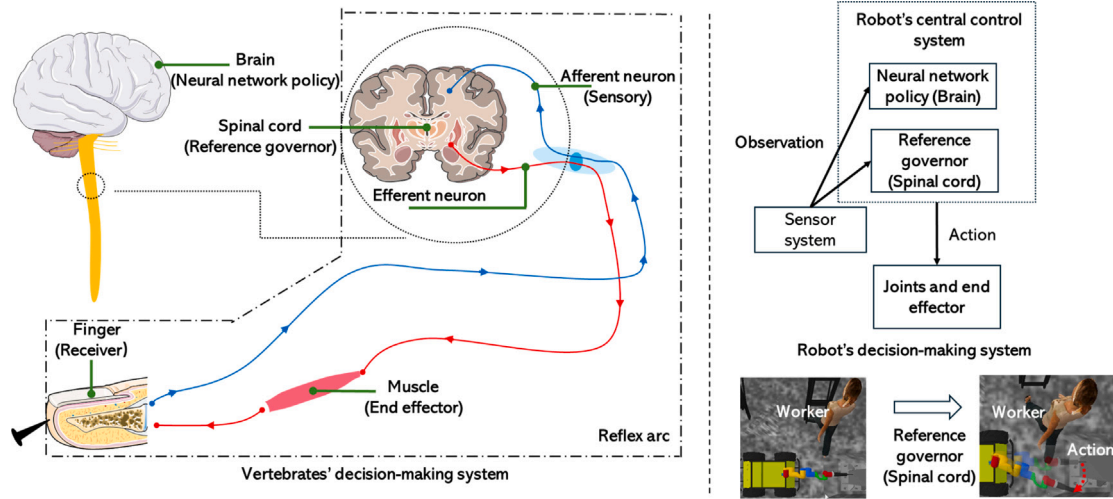


Fig. 1. Inspiration for the proposed control method.

often require human operators to assist or control the robots, and accidents have been reported when the robots were switched on [13]. Furthermore, there has been a lack of studies on construction robot control to meet the requirements of SSM. As a result, this paper aims to provide robots with a “SSM safe mode” during construction HRC, which enables close collaboration with human workers while ensuring robot efficiency [10,11].

Recent advances [14–16] in construction HRC safety primarily focus on pose estimation of workers and construction equipment, and predicting worker trajectories. These efforts fall within the domain of robot perception. Here, perception refers to robots recognizing workers and other surrounding objects, enabling the control system to then generate executable actions for robot actuators.

For construction equipment that is essentially a moving platform, i.e., Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs), conventional trajectory planning methods work well since they only have one rigid body [17]. However, complex construction equipment such as excavators and cranes usually contains multiple links. Task planning in a continuous, high-dimensional state-action space for multi-link robots is challenging, particularly given the uncertainty and time-varying nature of on-site construction environments and the long sequential nature of construction tasks. Conventional control methods [18,19] for construction robots have historically depended on pre-defined target positions and orientations to guide the robot's actions. While these methods can be effective when task parameters are predetermined, they often fall short in long sequential construction processes where the goal position and orientation must be determined dynamically at each step based on task progression and environmental constraints. Addressing this limitation requires combining these methods with learning-based approaches [20], which can dynamically generate intermediate goals, or alternatively relying on manual programming and operator intervention. Leveraging the approximation capabilities of neural networks, they used neural networks to predict the cost associated with specific actions [21]. By estimating this cost, robots can iteratively optimize a sequence of future actions in continuous, high-dimensional action spaces, enabling the selection of the optimal immediate action to generate the control sequence [22]. However, even with the integration of learned neural networks for prediction, recent advancements in methods such as differentiable MPC [21] still require solving an online optimization problem that involves neural networks, which means they can only adopt simple neural network architecture due to the computational and real-time implementation challenges. In the construction industry, robots are expected to incorporate advanced control architectures that

facilitate robust generalization across diverse tasks and dynamic environments while ensuring rapid responses to emerging safety risks. To this end, researchers [23,24] defined construction task planning as a Markov Decision Process (MDP) problem and turned to deep reinforcement learning (DRL) methods for direct inference instead of online optimization.

DRL methods directly update neural network policies through trial-and-error using an optimizer to maximize accumulated rewards, enabling the robot to focus on optimizing long-term performance in long construction sequences. The neural network policies receive sensory observations from the robot's sensors and directly output actions through forward propagation, offering a more efficient approach by eliminating the need for online optimization. DRL has proven to be efficient in acquiring complex construction skills, such as collaborative installation [23]. [25] demonstrated that DRL algorithms can achieve rewards more than 5 times higher than conventional control methods. On the other hand, penalties can be imposed for unwanted effects, such as collisions between the robot and workers. In other words, if a collision occurs, the robot receives a penalty cost during training, motivating the learned policy to prioritize avoiding collisions. By defining the cost, the problem can be reformulated as a maximization problem under constraints, enabling the development of primal-dual-based methods to solve it. The final policy can theoretically satisfy the constraints given sufficient training data. However, a critical challenge in applying DRL-based control system to construction robots lies in the lack of structured, large-scale training datasets that include safety-critical scenarios (e.g., robot failures during high-speed movements) and out-of-distribution situations (e.g., novel worker behaviors not seen during training). This limitation hampers the ability of DRL models to generalize effectively and ensure safety in real-world HRC operations.

Despite advancements in DRL for robotic control, existing DRL-based control systems also lack rigorous frameworks to provide theoretical safety guarantees, particularly in HRC within construction environments. The absence of formal safety assurances limits the practical deployment of DRL-based systems in high-stakes construction tasks.

In addition, existing methods emphasize reactive collision avoidance while overlooking the incorporation of joint-level constraints, such as angular limits, velocity bounds, and torque restrictions in the control system. For instance, if a neural network policy outputs joint angles, incorporating torque restrictions becomes challenging. Conversely, if the neural network policy directly outputs joint torques, proving the system's asymptotic stability remains an unresolved issue. The joint-level constraints are essential for ensuring the robot's

operational safety, preserving its structural integrity, and maintaining reliable performance in dynamic and safety-critical construction environments. This gap is particularly significant in the construction industry, where high-power, heavy-load machinery—such as hydraulic systems—faces unique challenges. These systems are prone to issues like overshooting, oscillations, and heat buildup, which can degrade hydraulic oil and seals, as well as increased energy dissipation through valves or mechanical means, especially compared to electric-driven robot systems.

Therefore, the current research gaps in safe HRC calls for the development of new methods to solve the following problems:

- How can DRL-based control systems, trained without large-scale high-quality datasets, effectively handle safety-critical and out-of-distribution scenarios in construction HRC?
- How can DRL-based control systems provide theoretical safety guarantees for construction HRC safety?
- How can DRL-based control systems avoid collisions with the worker while consider constraints on joint angles, velocity, and torques?

To address these, this paper draws inspiration from the central nervous system of vertebrates (see Fig. 1) that contains a brain and a spinal cord [26]. In specific situations, especially those concerning safety, the muscle actuator first receives rapid responses from the spinal cord before instructions from the brain. This capability arises because the spinal cord serves as a center for coordinating various reflexes and contains reflex arcs that can respond independently [27]. For instance, when a person's finger is pricked by a nail, the finger is instinctively withdrawn (see Fig. 1). Similarly, during the patellar reflex, the lower leg kicks forward involuntarily, even if the person does not intend to move it. Another example is pupil light reflex, in which the pupils autonomously constrict or dilate in response to light intensity changes. This protective reflex ensures a rapid response to environmental changes to protect eyes.

Drawing from this inspiration, we aim to design a biomimetic control method to improve current DRL-based control for construction so that the robot can intuitively respond to stimuli and ensure safety during HRC. Analogous to vertebrates' central nervous system, the robot's control system comprises both a brain (neural network policy) and a spinal cord (reference governor layer) (see Fig. 1). This reference governor layer is defined by three key properties: **safety guarantee, fast response, and compatibility with the neural network policy**. To elaborate, first, the layer must be explainable and provide a theoretical safety guarantee to meet the reliability requirements of the construction industry. Second, the layer must respond rapidly to stimuli in human–robot interactions, avoiding time-intensive inference and online optimization. This need is becoming increasingly critical as current neural network policies for handling complex tasks grow in complexity, demanding significant computational resources for inference. For instance, state-of-the-art text-guided policies based on diffusion models can take up to 4 s to plan a trajectory [28]. In this time, a worker could move a significant distance, potentially introducing safety risks due to the delay in trajectory updates. Finally, the layer should seamlessly integrate with the neural network policy to form a control system. While the neural network policy excels in learning and executing complex construction tasks, this control layer is dedicated to managing intuitive reactions specifically for construction safety.

The robot's intuitive safety control layer is based on the reference governor theory derived from vector field methods in model predictive control (MPC) [29]. The vector field is an add-on unit that filters the desired reference, ensuring that the system's trajectory consistently meets all constraints, while avoiding the need of complex online optimization. Specifically, the reference governor layer (i.e., the intuitive safety control layer) can directly modify the reference actions generated

by the neural network policy, performing intuitive reactions for the robot. Previous implementations of reference governors on multi-link robotics either ignore collision avoidance [30], or lack a brain-like neural network policy, resulting in systems that only have intuitive reactions [31]. In contrast, our system proposes a biomimetic control method for safe construction HRC. In this system, both input saturation (i.e., maximum torque or velocity for an actuator) and collision avoidance are considered. The effectiveness of the proposed system is evaluated through seven simulation and real-world robot experiments. Our contribution includes:

- We propose a biomimetic safety-constrained DRL control method. To the best of our knowledge, we are the first to provide theoretical safety guarantees for DRL-based control in construction HRC, which bypasses the black box nature of neural network policies. The proposed method ensures safety even when the neural network policy encounters out-of-distribution scenarios in on-site construction.
- To the best of our knowledge, this is the first safe DRL study to offer comprehensive dynamic safety in construction HRC, encompassing distance, torque, and velocity constraints based on robot dynamics. This approach ensures the actual robot motion adheres to safety standards, addressing key limitations in current DRL-based control methods, which often overlook dynamic issues such as overshoot, oscillations, heat buildup, and energy dissipation due to excessive velocities.
- We eliminate the need for designing complex DRL reward functions to explicitly consider HRC safety. With our method, the reward function does not need to consider safety constraints such as collision avoidance and input saturation. Therefore, training of the neural network policy can be more efficient.
- The proposed method ensures rapid response to safety risks, akin to the spinal cord's reflexive function, by reacting to environmental stimuli within 0.01 s. With an average computation time of 0.00565 s for complex safety constraint planning, it effectively decouples safety response from complex construction task planning. This enables the method to support advanced neural network architectures while maintaining rapid safety responses.

2. Background

2.1. Reinforcement learning in construction robotics

DRL-based control for construction robots has been extensively studied in recent years [32,33]. Most studies in this area aim to harness the exceptional learning capabilities of neural networks to enhance work efficiency, address dynamic construction environments and complex construction tasks.

DRL improves work efficiency since it has the potential to offer faster inference speeds compared to human workers. For instance, [34] demonstrated that construction hoist control using a deep Q-learning policy outperformed human control in terms of efficiency. Similarly, [35] showed that DRL could achieve higher efficiency in operating construction machines compared to even expert human operators.

In addition, DRL has demonstrated its capability in learning control policies tailored for dynamic construction environments. Such environments often involve complex interactions among multiple agents or changes in robot dynamics due to varying loads. For instance, [23] proposed a multi-agent proximal policy optimization (PPO) approach to coordinate multiple agents in collaborative tasks. Their method achieved a success rate exceeding 90% with a low collision rate ($\leq 6\%$ per task) during installation operations.

Furthermore, unlike other control methods, DRL offers a straightforward implementation due to its model-free nature, allowing it to adapt to the dynamic and kinematic properties of construction robots. For instance, [36] showcased DRL's ability to adapt to fluctuations in robot

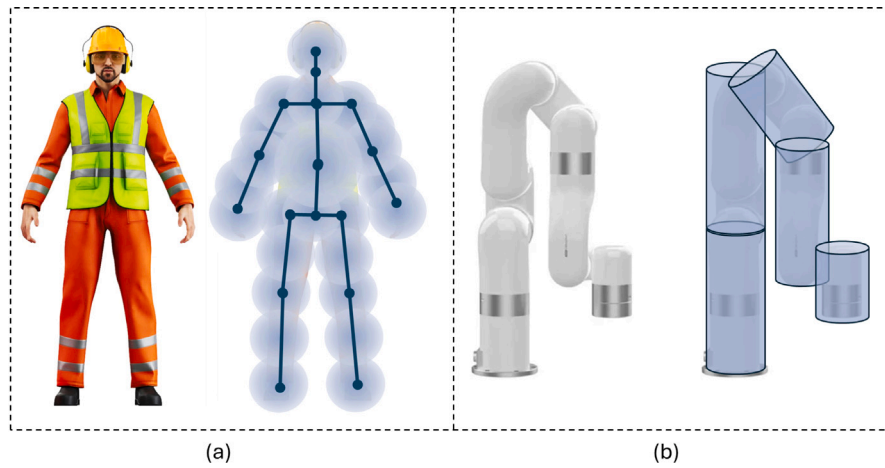


Fig. 2. Parametric models describe the space occupancy of workers and robots: (a) Transparent blue spheres, generated based on the blue skeleton, represent the space occupied by the body; (b) transparent blue cylinders enclose the robot to indicate its space occupancy.

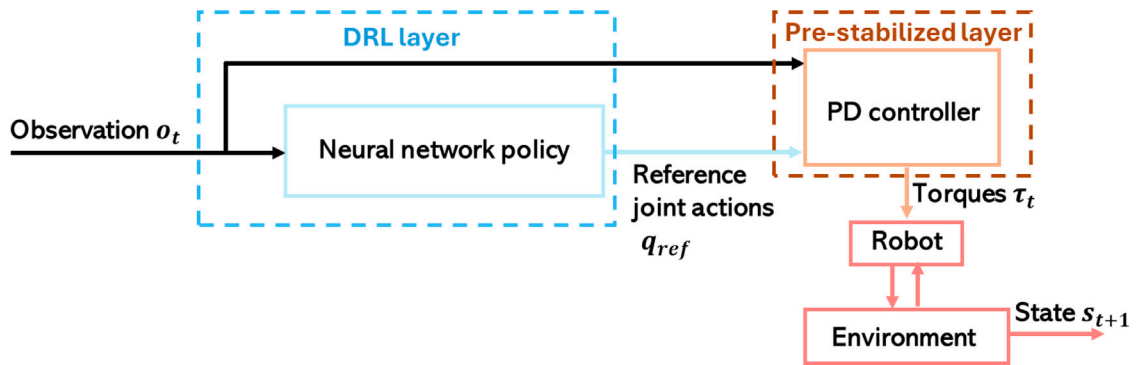


Fig. 3. Conventional two-step DRL decision-making process. The neural network policy generates the reference joint actions, and the low-level controller converts these joint actions into torques for execution. This type of control system relies entirely on the performance of the neural network policy.

dynamics induced by heavy external loads. [24] developed a robot evolution framework where DRL was utilized to control the robot for navigation within a heating, ventilation, and air conditioning system.

DRL also accelerates complex construction task learning and reduces the time required for parameter tuning compared to conventional control methods. For instance, [37,38] introduced DRL methods that can directly learn from expert demonstrations. Additionally, [39] observed that expert demonstrations may not always be the most efficient and proposed a methodology to initially learn basic skills from demonstrations and subsequently improve these learned skills.

Despite recent progress in implementing DRL in construction, the control policy from a DRL agent is often perceived as a black box model lacking transparency. This is problematic in industries where reliability is paramount. Especially in construction, where machines handle high payloads and torques, ensuring control safety is critical. In this regard, learning-based models struggle to guarantee safety, even with intricate reward functions specifically designed for collision avoidance. This problem is acute particularly when faced with out-of-distribution data during execution. Therefore, this study aims to tackle these challenges and establish safety assurance for our control method while leveraging the advantages of neural network policies.

2.2. Safe human–robot collaboration in construction

Ensuring safe HRC in construction environments is crucial due to the inherent risks posed by heavy machinery and the dynamic workspace. Potential safety challenges of HRC includes physical risks, psychological risks, and attention costs [5], with physical risks being the most prominent.

To reduce physical risks during construction HRC, researchers developed methods to enhance machine perception. For example, computer vision models can be applied for interpreting construction scenes and estimating positions of both workers and robots [16]. [40] introduced a deep learning-based model that enables robots to understand semantic contexts in scenes, thereby facilitating better understanding of facility relationships. [41] further utilized computer vision techniques to monitor distance between workers and robots through live videos, ensuring safety during collaborative tasks. [15] proposed a marker-less vision model to accurately estimate the 3D positions of construction robots' joints, enhancing spatial awareness and operational precision, leading to more efficient collision avoidance. In addition, researchers have explored modifying construction machine trajectories based on predictions of worker movements [42]. For instance, [17] introduced an uncertainty-aware long short-term memory network to forecast worker movements and employed a DRL-based method to generate robot actions accordingly. However, in these studies, HRC safety heavily relied on the training performance of the DRL agent.

Early studies also aimed to investigate worker's attention in HRC. For example, [43] found the collaboration with small ground robots may lead to lower physical demand in workers' attention. [14] employed immersive virtual environments to assess workers' attention to safety while working alongside robots, analyzing workers' readiness to engage in such collaborations. Another effective strategy to enhance HRC safety involves worker training. For example, [44] developed virtual environments to train workers for effective collaboration with robots, aiming to improve task efficiency and safety.

However, there is a notable gap in the research of robot control in safe HRC. While DRL methods can achieve collision avoidance by

integrating penalties into reward functions, they lack interpretability and cannot provide theoretical guarantees. Moreover, DRL's two-step control approach (detailed in Section 3.1) does not adequately address the dynamic properties of construction robots in low-level control. Therefore, this paper focuses on improving construction robot control and solving the aforementioned issues in DRL-based control method.

2.3. Safe deep reinforcement learning

While research on robot safety has a long history, the development of safe DRL algorithms remains nascent. Most approaches consider safety as a set of cost constraints, with the DRL objective to minimize these costs while maximizing cumulative rewards [45,46].

While constraint violations are commonly treated as negative costs and combined with rewards, separating the cost function offers distinct advantages by enhancing interpretability and facilitating analysis of constraint adherence during training [47]. By defining accumulated rewards and costs, one can develop methods such as primal-dual-based methods [48,49] to optimize cumulative rewards and costs. The trade-off between rewards and costs can be managed through functions like the Chernoff bound [50].

Constraints can also be learned by a neural network, as in [51,52], where a conservative safety critic is used to estimate safety. However, such methods may still permit occasional unsafe actions that could compromise critical robot applications. More rigorous safety guarantees have been explored using conditional value-at-risk constraints rather than expectation-based constraints, which offer improved robustness [53,54].

Most existing safe DRL applications, however, are limited to simulations with oversimplified environments and basic safety constraints [55], such as obstacle avoidance of spheres in static environments with cylinder obstacles [56,57]. In this study, we address safe DRL in the context of multi-link robot applications in onsite construction. This work addresses not only physical collision avoidance but also control safety constraints, including limits on joint angles, velocities, and torques to prevent potential robot damage. These comprehensive but much more complex constraints align better with engineering applications, particularly under standardized safety specifications such as SSM mode.

Furthermore, many approaches rely on learned models to approximate robot dynamics and enforce safety constraints [58]. However, these models are often limited by training data and may perform poorly in out-of-distribution scenarios. Ensuring correct operation during the learning process remains an unresolved challenge, especially for safety-critical applications [59]. In this study, we integrate the robot's actual dynamics into our safety framework to enhance robustness and ensure stricter safety adherence under SSM mode.

3. Preliminaries and problem formulation

In this section, we present a potential collision problem between workers and robots caused by the neural network policy's black box property in construction robot control.

3.1. Problem statement

Consider a scenario where the workers and robots share the same workspace; the robots are required to adhere to the SSM mode and other safety constraints under all circumstances.

The SSM mode specifies that robots should monitor the distance to nearby workers and adjust their speed or even stop work if necessary, based on the safety constraints. In SSM mode, the robots continue working if there is sufficient safety, dynamically adjust their motion as the safety conditions change, and will ultimately stop if necessary (e.g., if a collision is unavoidable due to the worker's high speed or the robot motors' maximum power).

Two-step control system. Current DRL control methods for controlling nonlinear robot systems to complete a construction task are essential two-step methods. First, a trajectory $\mathcal{T} = \{q_t \in \mathbb{R}^n \mid t \in [t_0, t_f]\}$ is generated for a n DoF robot by a neural network policy π_θ . Second, corresponding torques $\tau = \{\tau_t \in \mathbb{R}^n \mid t \in [t_0, t_f]\}$ are calculated by a pre-stabilized layer (e.g., PID controller) for the fully actuated system, as shown in Fig. 3. With τ , the robot executes the construction task, resulting in the trajectory $\mathcal{T}_{gt} = \{\tilde{q}_t \in \mathbb{R}^n \mid t \in [t_0, t_f]\}$ during the execution process. In real applications, $\mathcal{T}_{gt}$ and $\mathcal{T}$ are not identical during the robot dynamic properties.

The discrepancy between $\mathcal{T}_{gt}$ and $\mathcal{T}$ can be attributed to several factors, with the primary contributing factor being the overshoot that occurs within the pre-stabilized layer. To elaborate, in a discrete-time robot control system, the system receives the reference joint angles $q_{ref} \sim \pi_\theta$ at time step t . Assuming q_{ref} remains constant and satisfies safety constraints, there is still potential for constraint violations during systems' convergence to q_{ref} . This occurs because $\tilde{q}_t$, influenced by τ generated by the pre-stabilized layer, may temporarily exceed q_{ref} as they converge to q_{ref} [60].

Problem setup. For the above control system, the target can thus be formulated as obtaining τ that can control a robot to complete construction tasks while $\mathcal{T}_{gt}$ is expected to adhere to safety constraints $C = \{\tilde{q}_t \in \mathbb{R}^n, oc_t \in \mathbb{R}^{k \times 3} \mid \mathcal{G}(\tilde{q}_t) \leq 0, \mathcal{H}(\dot{\tilde{q}}_t) \leq 0, \mathcal{F}(\ddot{\tilde{q}}_t) \leq 0, \mathcal{O}(\tilde{q}_t, oc_t) \leq 0, t \in [t_0, t_f]\}$ during task execution. $\mathcal{G} : \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathcal{H} : \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathcal{F} : \mathbb{R}^n \rightarrow \mathbb{R}$, and $\mathcal{O} : \mathbb{R}^n \times \mathbb{R}^{k \times 3} \rightarrow \mathbb{R}$ denote constraints related to $\tilde{q}$, $\dot{\tilde{q}}$, $\ddot{\tilde{q}}$, $\tilde{q}$ and oc respectively. oc include k points to represent the workers' space occupancy.

$\mathcal{T}$ is steady-state admissible if it can generate τ such that C holds for all time instances. C encompasses a comprehensive range of safety conditions applicable to the robot, including:

- Joint angle constraints: This includes constraints on joint angles, ensuring that the joint angles do not exceed specified limits:

$$\mathcal{G}(\tilde{q}) \leq 0 \quad (1)$$

- Velocity constraints: This encompasses constraints on both joint velocities and end-effector velocities, ensuring that the velocities do not exceed specified limits:

$$\mathcal{H}(\dot{\tilde{q}}) \leq 0 \quad (2)$$

- Acceleration (Torque) constraints: These constraints ensure that the acceleration of the joints, which can be related to the torques applied, remains within safe limits to prevent mechanical overload:

$$\mathcal{F}(\ddot{\tilde{q}}) \leq 0 \quad (3)$$

- Distance constraints: These constraints ensure that the distances between workers and robots remain within safe limits, effectively preventing collisions:

$$\mathcal{O}(\tilde{q}, oc) \leq 0 \quad (4)$$

Human space occupancy. Space occupancy of the worker's body is denoted by a parametric human body model. In this study, inspired by the 3D Gaussian parametric hand model proposed by [61], we used a similar parametric body model to represent the human body space occupancy, as shown in Fig. 2(a). The model can be denoted by a set of points $H = \{(x, y, z) \in \mathbb{R}^3 \mid \min_{i \in \{1, \dots, k\}} \|(x, y, z) - c_i\| \leq r_i\}$. H represents the set of points in k closed balls centered around the key points c_i for $i = 1, 2, \dots, k$. Generally, we have $c_i \in oc$; specifically, at time t , we have $c_{i,t} \in oc_t$.

Robot space occupancy. This study will focus on a serial-link robot equipped with a single end effector. For the collaborating robot, given $\tilde{q}$, we can compute the positions of links via forward kinematics $fk(\tilde{q})$. The position of all links can then be used to generate several cylinders

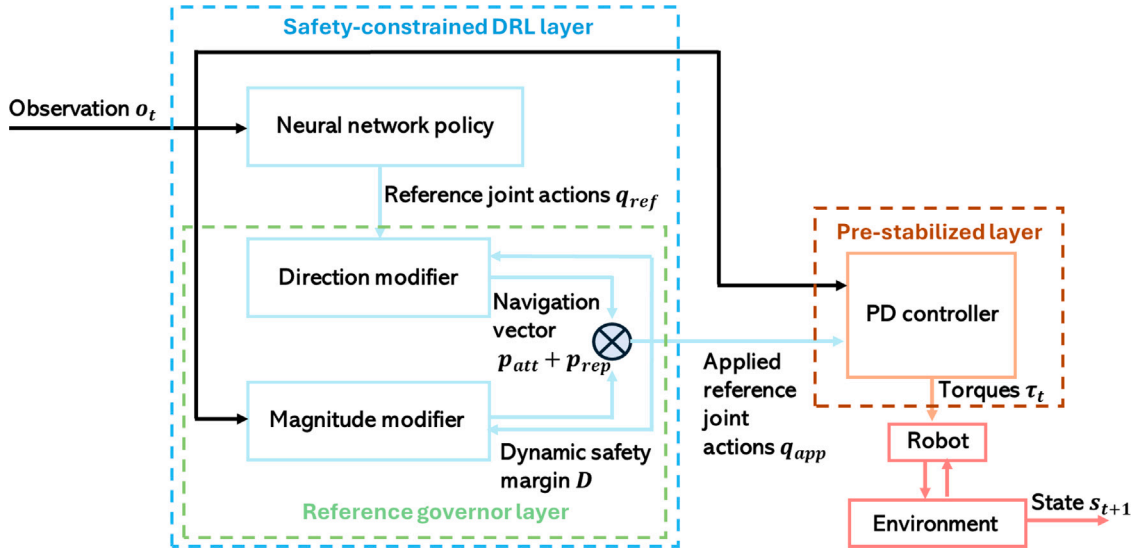


Fig. 4. Proposed decision-making process for HRC safety.

that can include the robot arm. Assume the set of points on m cylinders is $H = \bigcup_{k=1}^m \{(x, y, z) \in \mathbb{R}^3 \mid x^2 + y^2 \leq r_k^2 \text{ and } z \in [h_{k1}, h_{k2}]\}$. r_k is the manually defined radius of cylinder k , and $[h_{k1}, h_{k2}]$ defines the length, which can be computed based on $f_k(\hat{q})$. In the definition, the coordinates (x, y, z) are expressed in the local frame associated with each cylinder.

Dynamic safety constraints. Since $\mathcal{T}_{gt}$ and $\mathcal{T}$ are not identical, the control system must account for the robot's dynamic properties to ensure that Eqs. (1)~(4) hold during task execution. In this study, dynamic safety margin (DSM) was used to denote the safety redundancy during actual robot motion.

3.2. Safety concerns regarding deep reinforcement learning

DRL has shown promising results in collision avoidance and safety control. However, current DRL methods are still far from being applicable in industry due to concerns about their reliability.

In a DRL problem, at time t , the state of the construction environment is s_t , the observation of the robot is o_t , action generated by the neural network control policy π_θ at state s_t is q_t , and the reward generated by taking q_t in single step is r_t , the expected long-term reward from a given state s_t is $V(s_t)$, the expected long-term reward from s_t by taking q_t and following the policy π_θ is $Q(s_t, q_t)$. In this study, the environment can be considered fully observable since the robot can obtain all necessary information (e.g., the states of obstacles and its states) from its sensors to complete tasks. DRL focuses on maximizing the accumulated rewards during task execution:

$$\sum_{t=0}^T \gamma^t r_t(s_t, q_t) \quad (5)$$

where γ is the reward discount factor, and T is the time horizon. In a two-step control system, q_t is simply used as q_{ref} (see Fig. 3). DRL maximizes Eq. (5) by exploring the environment and updates the policy π_θ according to the reward received. The DRL agent approximates the control policy through neural networks. Due to the generalization ability of neural networks, if a safety penalty term is incorporated to the reward computation, the neural network can theoretically learn a good policy π_θ to avoid violating safety constraints after adequate exploration.

However, quantitatively defining adequate exploration remains challenging, and ensuring safety for DRL control methods under out-of-distribution observations o_t is difficult due to the black box property of neural networks. This is particularly concerning for DSM, since the DRL

agent must adeptly acquire a precise understanding of the robot system's dynamic properties across all conditions. Hence, using only DRL to meet safety constraints across various construction environments poses significant challenges.

3.3. Research objective

Based on the above discussion, to address the problem proposed in Section 3.1, this study aims to propose an advanced DRL control system and provide theoretical safety guarantees for the proposed method.

4. Methodology

Generally, in a two-step control system, the update frequency of the reference joint actions q_{ref} from a neural network policy is significantly slower than that of the pre-stabilized layer. This can improve computational efficiency and stabilize policy learning [62]. Low update frequency can also result from the slow inference speed of complex neural network models [28]. In dynamic HRC environments, this can hinder the control system's ability to respond promptly to safety-critical stimuli.

To satisfy safety constraints (Eqs. (1)~(4)), the trajectory $\mathcal{T}$ supplied to the pre-stabilized control layer must be steady-state admissible. Inspired by the vertebrate central nervous system, this study decouples safety adherence from task learning by introducing a high-frequency (e.g., 100 Hz) reference governor layer. This layer modifies trajectories $\mathcal{T}$ in real time, enabling complex neural network policies to handle challenging tasks while ensuring $\mathcal{T}$ remains steady-state admissible and enabling rapid responses.

The reference governor layer operates in parallel with the neural network policy as a safety-critical component, capable of functioning independently to manage safety risks. Initially, before receiving any reference action array from the neural network policy, the reference governor layer sets the robot's current state as its default reference. This ensures that when the robot is idle, it remains in its original state while the reference governor layer actively prevents collisions. During operation, the reference governor layer updates its reference based on action arrays sent by the neural network policy. Operating at high frequency, it maintains the current reference until an updated array is received.

An overview of the proposed control system is illustrated in Fig. 4. First, observations from the RGB-D camera capture the state used for decision-making. The neural network policy, initially trained without

safety constraints, generates reference joint angles q_{ref} . These angles undergo adjustment by a vector field computed from direction and magnitude modifiers. The resulting modified joint angles, which ensure the robot will strictly adhere to safety constraints, serve as the final outputs of the safety-constrained DRL layer. In the second step, these modified joint angles are subsequently used as input for the PD controller to generate torques for actuators.

4.1. Pre-stabilized control layer

Before discussing the safety-constrained DRL layer, it is essential to introduce the dynamic model in the pre-stabilized layer for DSM calculations. In this study, a classic PD controller with gravity compensation was developed in this layer. The dynamic model of a construction robot with n joints can be calculated by:

$$\tau = \mathbf{M}(q)\ddot{q} + \mathbf{C}(q, \dot{q})\dot{q} + \mathbf{G}(q) \quad (6)$$

where τ is the vector of joint torques, $\mathbf{M}(q)$ is the inertia matrix, $\mathbf{C}(q, \dot{q})$ is the matrix representing Coriolis and centrifugal forces, and $\mathbf{G}(q)$ is the gravity vector. Instead of using q_{ref} , we define q_{app} as the applied reference joint actions (i.e., the actual joint actions that the low-level PD controller needs to execute) sent to the pre-stabilized control layer. The PD controller's objective is to calculate the corresponding torques:

$$\tau = \mathbf{K}_p(q_{app} - q) - \mathbf{K}_D\dot{q} + \mathbf{G}(q) \quad (7)$$

where $\mathbf{K}_p$ and $\mathbf{K}_D$ are the proportional and derivative gain matrices, respectively. $\mathbf{K}_p$ adjusts the torque based on the position error, while $\mathbf{K}_D$ adjusts the torque based on error's derivative. The globally asymptotically stability of the low-level PD controller can be analyzed with the Lyapunov function.

$$V(q, \dot{q}, q_{app}) = \frac{1}{2}\dot{q}^T \mathbf{M}(q)\dot{q} + \frac{1}{2}(q_{app} - q)^T \mathbf{K}_p(q_{app} - q) \quad (8)$$

If $\mathbf{K}_p$ and $\mathbf{K}_D$ are positive definite diagonal matrices, the equilibrium configuration of Eq. (8) can be proven to be globally asymptotically stable according to Lyapunov's second method [63].

4.2. Safety-constrained deep reinforcement learning

In the safety-constrained DRL layer, the DRL agent receives an observation as the input and generates the applied reference joint actions q_{app} for the pre-stabilized layer as the output. At a high level, the neural network policy in this layer first generates the reference joint angles q_{ref} to guide the robot to the desired set point based on specifications of the construction task. This q_{ref} is generated without consideration of safety constraints. Then, the safety constraints form a navigation vector field via the direction modifier to alter the direction of the robot's movement. The dynamic model is further utilized to compute the DSM vector, which describes the magnitude of the modification as a magnitude modifier.

To comprehensively introduce the safety properties of the layer, we consider the worst-case scenario in this section, where the initial DRL policy does not incorporate any safety constraints into its reward function.

Neural network policy. In this study, a neural network policy π_θ is used to generate the reference joint actions $q_{ref} = \pi_\theta(s_t)$. Policy's parameters are updated based on rewards received during exploration. The update strategy adopted is policy gradient:

$$\nabla J(\theta) \propto \mathbb{E}_\pi [Q_\pi(s, q_{ref}) \nabla_\theta \ln \pi(q_{ref} | s, \theta)] \quad (9)$$

where $Q_\pi(s, q_{ref})$ is the expected accumulated rewards that an agent can receive at state s by executing action q_{ref} under policy π_θ ; $\pi(q_{ref} | s, \theta)$ is the probability of taking q_{ref} in s under π_θ . To improve learning stability, $Q_\pi(s, q_{ref})$ in Eq. (9) is replaced by an advantage

function $\hat{A}$. In this paper, we compute the advantage function $\hat{A}$ via generalized advantage estimation (GAE), as described in [64]:

$$\hat{A}_t = \delta_t + (\gamma\xi)\delta_{t+1} + \dots + (\gamma\xi)^{T-t-1}\delta_{T-1} \quad (10)$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (11)$$

where ξ is a discount factor in GAE.

Control the similarity between the old policy and the updated policy is another strategy used in this paper to improve the learning stability. We define a parameter ζ to represent the clip coefficient aiming to limit the value of $\frac{\pi_\theta}{\pi_{\theta_{old}}}$ between $1 - \zeta$ and $1 + \zeta$. This is essentially trying to avoid sudden changes in the control policy in each policy update. Therefore, Eq. (9) is replaced by the clipped loss function [65].

$$L(\theta) = \mathbb{E} \left[\min \left(\frac{\pi_\theta}{\pi_{\theta_{old}}} \hat{A}, \text{clip} \left(\frac{\pi_\theta}{\pi_{\theta_{old}}}, 1 - \zeta, 1 + \zeta \right) \hat{A} \right) \right] \quad (12)$$

The inclusion of the clipped loss function is primarily intended to enhance learning stability and is not intrinsic to the proposed control system.

In this study, we specifically avoid adding a negative reward (e.g., a reward of -1 whenever a collision between the worker and the robot is detected) for the DRL agent. This design is to showcase that our method provides effective safety guarantees even without integrating specific safety constraints into the reward function. By focusing solely on task completion, we simplify the reward function design, thereby enabling faster learning for the DRL agent.

Direction modifier. The direction modifier is tasked to generate a navigation vector field to modify the movement direction of the robot.

Attraction vector. First, we calculate the attraction vector p_{att} using the equation below. p_{att} guides the robot arm to q_{ref} .

$$p_{att} = \frac{q_{ref} - q_{app}}{\max(\|q_{ref} - q_{app}\|, \eta)} \quad (13)$$

where η is the smooth parameter. Since we use q_{app} to denote the applied reference joint actions in this paper, Eq. (13) essentially describes the direction from q_{app} to the reference joint actions q_{ref} generated by the neural network policy π_θ . In Eq. (13), the values of q_{ref} and q_{app} are from the current time step, and p_{att} uses these values to calculate q_{app} for the next time step.

Repulsion vector. Correspondingly, we can define the repulsion vector p_{rep} as:

$$p_{rep} = p_h + p_q \quad (14)$$

where p_h represents the repulsion vector from the worker's space occupancy, and p_q represents the repulsion vector from the joint limitations. p_{rep} therefore generates a repulsion to push the robot arm away from the steady-state inadmissible regions caused by the constraints of joint limitations and collision avoidance.

Based on the defined spatial occupancy of workers and robots, we introduce the nominal safety radius r_{ij} for each pair encompassing a closed ball j and a cylinder i . The radius r_{ij} provides a safety distance for the points in R and H . The radius r_{ij} should be greater than the sum of the radii of the ball and the cylinder, and may include additional safety margin depending on the requirements of the construction tasks. If a larger safety space is preferred, r_{ij} can be assigned a larger value. Then, we can calculate the minimum distance vector d_{ij} between them as:

$$d_{ij} = p_i + \hat{\lambda}_{ij}(p_{i+1} - p_i) - c_j \quad (15)$$

where link i is defined by the joint positions $p_{i+1} - p_i$, and the coefficient $\hat{\lambda}_{ij} \in [0, 1]$ can be calculated as:

$$\hat{\lambda}_{ij} = \frac{(p_{i+1} - p_i)^T (c_j - p_i)}{(p_{i+1} - p_i)^T (p_{i+1} - p_i)} \quad (16)$$

Next, we can define a coefficient $\hat{d}_{ij}$ to represent the risk level for any distance:

$$\hat{d}_{ij} = \max \left(\frac{\delta_{RH} - (\|d_{ij}\| - r_{ij})}{\delta_{RH} - o_{RH}}, 0 \right) \quad (17)$$

where o_{RH} is used to control the scale of $\hat{d}_{ij}$. When the minimum distance between the points in R and the points in H is less than δ_{RH} , the robot starts to react to worker's movement. Since d_{ij} describes the vector in the Cartesian space, we can calculate the corresponding angles in joint action space via the pseudo-inverse Jacobian $J^\dagger$ as:

$$\hat{p}_{h,ij} = \frac{J^\dagger(q)d_{ij}}{\max(J^\dagger(q)d_{ij}, \eta)} \quad (18)$$

Therefore, the repulsion vector $p_{h,ij}$ for sphere j and link i can be calculated by Eq. (19).

$$p_{h,ij} = \hat{d}_{ij} \hat{p}_{h,ij} \quad (19)$$

Since Eq. (18) defines a unit vector. In the safe cases, $\hat{d}_{ij}$ should be zero. If the distance is less than the safety margin δ_{RH} , the repulsion vector $p_{h,ij}$ starts to push the link i away from the worker. Further, if the distance between them is less than o_{RH} , the unit vector $\hat{p}_{h,ij}$ is amplified by $\hat{d}_{ij}$, and this denotes a high safety risk.

Similarly, we can also define the joint safety margin δ_q and the scale factor o_q . Then, we can calculate the $p_{q,i}$ for joint i 's limitation:

$$p_{q,i} = \max \left(\frac{\delta_q - \|q_i - q_{min,i}\|}{\delta_q - o_q}, 0 \right) - \left(\frac{\delta_q - \|q_i - q_{max,i}\|}{\delta_q - o_q}, 0 \right) \quad (20)$$

Eq. (20) is essentially computing the repulsion effect of the limitation of joint i on robot's direction.

Magnitude modifier. Through the magnitude modifier, we calculate the magnitude of the applied modifications considering safety constraints (Eqs. (1)~(4)).

Robot dynamics. In the above dynamic system, if we know the current joint angles q and the applied reference joint actions q_{app} sent to the PD controller, we can estimate the future robot movement using Semi-implicit Euler method. In a very short time interval Δt , we have:

$$\hat{\tau}_t = K_p(q_{app} - \hat{q}_t) - K_D\hat{q}_t + G(\hat{q}_t) \quad (21)$$

Next, the acceleration $\hat{q}_{t+\Delta t}$ can be estimated based on Eq. (6):

$$\hat{q}_t = M^{-1}(\hat{q}_t)(\hat{\tau}_t - C(\hat{q}_t, \hat{q}_t)\hat{q}_t - G(\hat{q}_t)) \quad (22)$$

Finally, we can compute the joint velocity and angles at $t + \Delta t$ as:

$$\hat{q}_{t+\Delta t} = \hat{q}_t + \hat{q}_t \Delta t \quad (23)$$

$$\hat{q}_{t+\Delta t} = \hat{q}_t + \hat{q}_{t+\Delta t} \Delta t \quad (24)$$

Starting from the current time t , we use $\hat{q}_t$ to approximate $\hat{q}_t$ and compute $\hat{q}_t$ over a sliding window of $n\Delta t$, ensuring adherence to safety constraints Eqs. (1)~(4) throughout this window. A sufficiently long window allows for DSM to be achieved during task execution.

Dynamic safety margin. DSM within the window needs to include limitations of torque, joint angle, joint velocity, end effector velocity, and safety distance, which is included in Eqs. (1)~(4). Eqs. (1)~(4) succinctly denotes the feasible region for control, it does not provide the computational detail needed for practical implementation. This part provides a set of explicit functions Eqs. (25)~(29) that contribute to making the robot system satisfy the overall constraints in Eqs. (1)~(4).

A set of points are sampled within the window, and the torque, joint angle, joint velocity, end effector velocity, and safety distance at these points are computed. Fig. 5 illustrates how the DSM is maintained during robot motion for joint angle constraints. At time t , there are q_{ref} from the neural network policy, q_{app} , and q . Based on q_{app} and q , future robot motion $\hat{q}_{t,t \in [t, t+n\Delta t]}$ under the pre-stabilized layer can be predicted. The dynamic safety margin is determined based on the

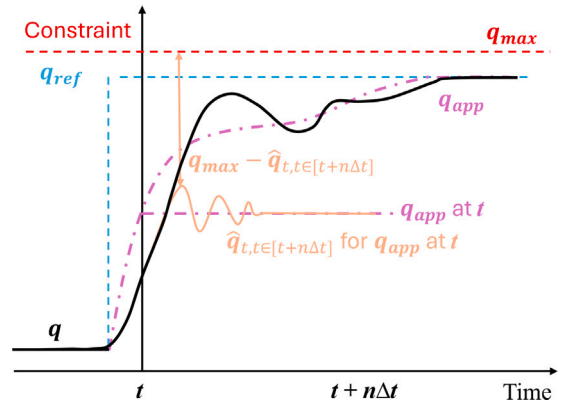


Fig. 5. Dynamic safety guarantee during robot motion.

minimum distance between $\hat{q}_{t,t \in [t, t+n\Delta t]}$ and the joint angle constraint q_{max} . If the dynamic safety margin is greater than zero, the derivative of q_{app} will not be zero. Otherwise, the derivative will be zero to prevent the robot from violating safety constraints in the future. In a scenario where a worker is moving towards the robot, a continuous decrease in the DSM will be observed.

The DSM of torque limitations can be expressed as:

$$D_\tau = k_\tau \min_{t' \in [t, t+n\Delta t]} \left(\min_{i=1,2,\dots,n} (\tau_{t',i} - \tau_{min,i}, \tau_{max,i} - \tau_{t',i}) \right) \quad (25)$$

As shown in Eq. (25), D_τ iterates through the sampled points within the interval $[t, t+n\Delta t]$ and the n joints to determine the torque value that is nearest to the torque limit. k_τ is a positive scaling factor that controls the scale of the D_τ . Similarly, the DSM of joint angle limitations can be expressed as:

$$D_q = k_q \min_{t' \in [t, t+n\Delta t]} \left(\min_{i=1,2,\dots,n} (q_{t',i} - q_{min,i}, q_{max,i} - q_{t',i}) \right) \quad (26)$$

where k_q is a positive scaling factor. The DSM of joint velocity limitations can be expressed as:

$$D_{\dot{q}} = k_{\dot{q}} \min_{t' \in [t, t+n\Delta t]} \left(\min_{i=1,2,\dots,n} (\dot{q}_{t',i} - \dot{q}_{min,i}, \dot{q}_{max,i} - \dot{q}_{t',i}) \right) \quad (27)$$

where $k_{\dot{q}}$ is a positive scaling factor. The DSM of the end effector velocity limitations can be expressed as:

$$D_{\dot{x}_e} = k_{\dot{x}_e} \min_{t' \in [t, t+n\Delta t]} \left(\min_{i=1,2,\dots,n} (\dot{x}_{e,t',i} - \dot{x}_{e,min,i}, \dot{x}_{e,max,i} - \dot{x}_{e,t',i}) \right) \quad (28)$$

where $k_{\dot{x}_e}$ is a positive scaling factor.

In addition, DSM with respect to the distance between the workers and the links must also be considered. If the link is far from the worker, it is safe to assign a large DSM. Conversely, if the link is close to the worker, the DSM should be reduced to satisfy SSM mode. This can be represented as:

$$D_{RH} = k_{RH} \min_{t' \in [t, t+n\Delta t]} \left(\min_{i=1,2,\dots,n} \left(\min_{j=1,2,\dots,k} (\|d_{ij}\| - r_j) \right) \right) \quad (29)$$

where k is the number of closed balls used to model the safety space surrounding the human body and other obstacles. k_{RH} is a positive scaling factor.

From Eqs. (25)~(29), the final DSM for each safety constraint will focus on the worst-case scenario. To improve the stability of the system, we do not want the direction modifier or the magnitude modifier significantly alter the joint actions in one step when the DSM is large. Therefore, the final output will be:

$$D = \text{clip}(\min(D_\tau, D_q, D_{\dot{q}}, D_{\dot{x}_e}, D_{RH}), 0, \delta_{th}) \quad (30)$$

where $\delta_{th} > 0$ is the threshold for D .

Finally, q_{app} can be updated by:

$$\hat{q}_{app} = D(p_{rep} + p_{att}) \quad (31)$$

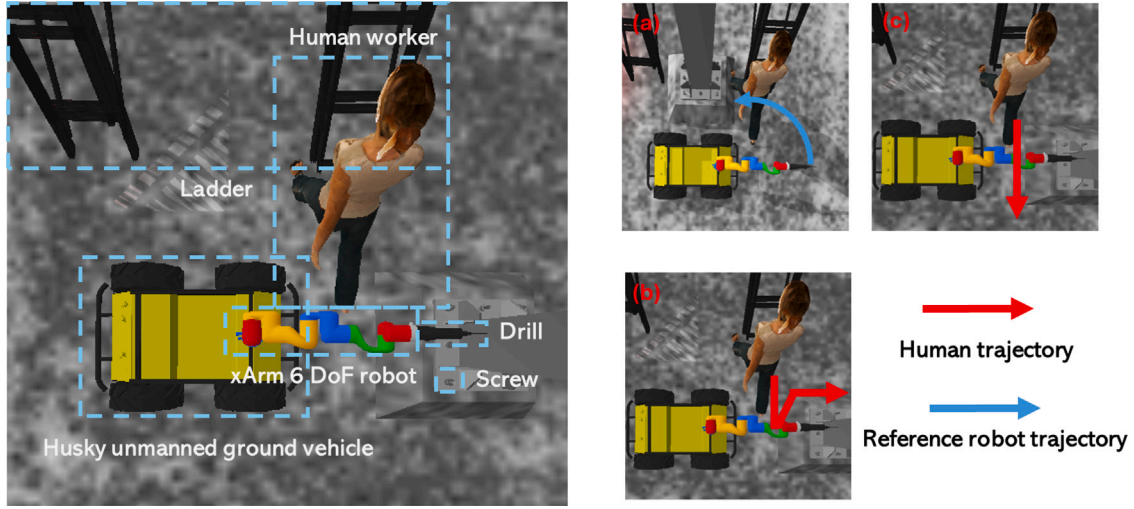


Fig. 6. HRC simulation environments (left) for evaluation: (a) Static configuration; (b–c) Dynamic configurations.

4.3. Safety analysis

This study decouples safety adherence from task learning by introducing a high-frequency reference governor layer. The neural network policy only focuses on specific construction tasks without consideration of safety constraints. Although the output of the neural network policy, i.e., the reference joint actions q_{ref} , can guide the robot to desired positions. q_{ref} must be modified before being sent to the PD controller. This modification aims to produce applied reference joint actions q_{app} that can satisfy safety constraints, and the final PD controller aims to provide a pre-stabilized control layer. The stabilization and safety of the control system are analyzed below.

First, q_{app} was sent to the PD controller at high frequency. However, q_{app} can be considered constant by the PD controller during a short time interval since the PD controller operates at much higher frequency (e.g., 1 kHz). Therefore, q_{app} will be an asymptotically stable equilibrium point for the controller [63].

Second, the direction of the movement of each link of a robot is determined by two vectors. The attraction vector points from its current applied reference joint actions to the reference joint actions, representing the direction to complete the task. The repulsion vector push each link away from the worker, and turn the direction of joints if they potentially violate safety constraints. With the direction modifier, we can alter the motion direction to avoid collisions and follow joint angle limitations. When the q_{ref} is steady-state admissible and $D > 0$, the control system is such that the equilibrium point $q_{app} = q_{ref}$ is asymptotically stable [66].

In addition, given that this is a two-step control system, and the low-level PD controller can overshoot during convergence, we propose a clipped DSM method. When the robot state is safe, it can apply high torques, and when the robot state approaches the boundary of safety constraints, it will reduce torques. If any safety constraint is about to be violated, the robot will stop its motion. It is important to note that each function for calculating DSM is continuous, as are the robot states and actions. Theoretically, given a robot trajectory $\mathcal{T}$ consists of q_{app} , the robot will meet all safety constraints if $D > 0$ along $\mathcal{T}$. Therefore, the robot can be guaranteed to not cause danger to the workers.

However, because of the fact that we assume the worker is static during the calculation of DSM, this may induce safety concerns in dynamic scenarios if the worker's speed is too fast. Assuming that the frequency of the modifiers is 100 Hz, if the worker's velocity is 1 m/s, then the worker can move 1 cm in each update of the modifier. Another problem is the limitation of the joint velocity. The system cannot avoid

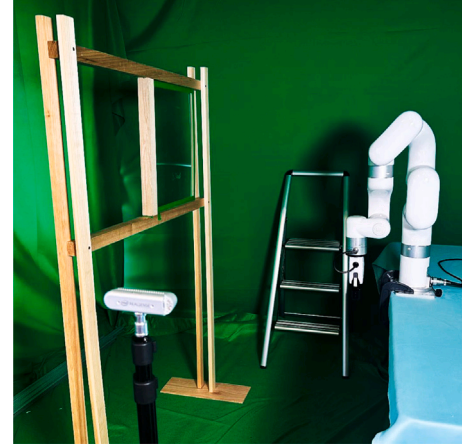


Fig. 7. Realistic HRC scenario for evaluation.

collisions once the relative speed of opposing movement is much faster than the robot's maximum speed. In such scenario, if the worker moves towards the robot very quickly, the collision cannot be avoided. With DSM, the control system can ensure that the robot halts reference updates if any safety violations could be found within a short future time window. This configuration satisfies the SSM mode.

5. Experiment design

5.1. Task overview

In this study, we consider a HRC scenario in which a worker and a robot arm operate in the same area. Therefore, when the worker focuses on a specific construction task, they might be unaware of the construction robot's actions due to noisy on-site environments, leading to potential injuries caused by the robot. In addition, the limitations of joint angles, velocities, torques, and the end effector velocity are defined according to the xArm robot user manual. Specifications on these parameters can be found in [Appendix](#).

When the worker is in the robot's working trajectory, they might remain stationary while the robot is moving, or they might move without being aware of the robot's actions. The former scenario is defined as a **static** HRC environment, while the latter is defined as a **dynamic** HRC environment. The term "static" refer to whether the worker's state

remains constant during robot operation. In both scenarios, we aim to evaluate whether the control system can correctly react to the worker's movements to ensure safety.

Simulation of the scenarios is illustrated in Fig. 6. In this simulation, the worker is coming down from a ladder and facing away from the robot, simulating a situation where the worker might be unaware of the robot's actions. The robot is mounted on a Husky unmanned ground vehicle, simulating a common construction robot configuration. The robot is equipped with a drill to perform screwing operations, which could potentially harm the worker. In this study, we fixed the position of the vehicle, so that we can better investigate if the proposed method can optimally control the robot arm to prevent injuries to the worker. The simulation environment was developed using Bullet Real-Time Physics Simulation [67]. The robot's configurations in the simulator, such as inertial parameters, damping, and friction, were set to the default values provided in the official URDF file. The simulation was run on a laptop-class i7-13620H CPU.

Based on this simulation, we designed three tasks. In the first task, the worker remains in the same position. In the second task, the worker moves towards the robot without crossing the robot's working trajectory. In the third task, the worker moves across the robot's working trajectory.

5.2. Static human-robot collaboration

- Task (a). In this task, the robot is following a working trajectory generated by the neural network policy (see Fig. 6(a)). The worker remains stationary within the robot's working trajectory. If the robot strictly follows this trajectory, the drill mounted on the robot arm will collide with the worker. This static scenario represents a situation where the worker is focusing on a construction task while not paying attention to the robot. The robot is expected to either stop at a safe distance or modify its trajectory to avoid injuring the worker.

5.3. Dynamic human-robot collaboration

In this scenario, the worker moves towards the robot arm without being aware of the robot's actions. The robot arm needs to maintain a safe distance to avoid collisions with the worker.

- Task (b). In this task, the worker will move a short distance towards the robot and then move away from it (see Fig. 6(b)). After the worker had moved away, the robot arm is expected to resume working.
- Task (c). In this task, the worker moves towards the robot arm and across the working trajectory without being aware of the robot arm's actions (see Fig. 6(c)). The robot arm needs to maintain a safe distance to avoid collisions with the worker and finally resume its work after the worker had moved away.

Task (b) and (c) represent situations where the worker enters the working trajectory of the robot, and the robot needs to modify its current trajectory in real-time.

5.4. Real human-robot collaboration experiment setup

The real robot test scenario is illustrated in Fig. 7. The scenario includes an xArm 6-DoF robot, a ladder, a RealSense D435i camera, and a window frame. Instead of being mounted on a vehicle, the robot arm is installed on a desk to perform operations on the window frame. The worker will turn their back to the robot arm. The 3D pose of the worker is estimated using the RGB and depth images via an off-the-shelf pose estimation tool box (e.g., Openpose [68]). The same three tasks were performed in this real-world HRC setup. The relative position between the workers, camera, and robot base can be obtained after camera calibration [69].

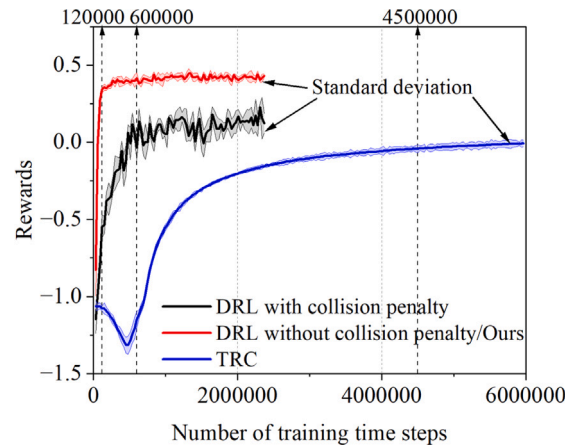


Fig. 8. DRL agent learning curves.

Furthermore, we conducted a final task, where the worker can perform *any actions* in this real-world HRC setup, and the robot had to react to these actions to ensure worker safety. This is because generating realistic 3D human motions in simulation is time-consuming. This task aims to evaluate the robustness of the proposed method across diverse worker poses. Compared to simulations, real robot tests offer a more accurate assessment of our control system's performance.

5.5. Training and evaluation configurations

All neural network models used in training shared the same architecture, learning parameters, and reward function, with an identical cost applied for safety constraint violations. For models without safety considerations, the corresponding cost term was omitted.

During training, the worker was not static; the initial position was randomized, and the worker could move randomly at each step. This randomization prevents the model from overfitting to specific movement patterns.

In evaluation, both initial positions and movement patterns were similarly randomized for each test. Detailed information on training and evaluation such as parameters, the designed values and functions for rewards and costs, and movement pattern parameters can be found in Appendix.

In evaluation, we compared the performance of the proposed method, conventional method, and the state-of-the-art DRL methods [53,65]. Collision was used as the safety index in these evaluations. For the simulation environment, the three tasks were tested for 100 times after training has finished, and the collision rate was recorded as the ratio of collisions to the total number of tests.

We evaluated six methods for a thorough comparison: (1) the proposed safety-constrained DRL method without a collision penalty in reward, (2) regular DRL with collision penalty in reward, (3) regular DRL without collision penalty in reward, (4) RRT, (5) TRC method in [53], and (6) model predictive path integral (MPPI). The neural network models adopted the same configurations except for their reward functions.

In method (4), RRT was employed for trajectory planning between the initial and goal states, with the goal state manually specified. Robot dynamics were integrated into the RRT process to predict actual motion and ensure collision-free trajectories in actual robot motion.

Method (5) used TRC, a state-of-the-art safe DRL method demonstrated on a real Jackal robot. Unlike other recent methods evaluated only in simple simulations, TRC shows stronger potential for real-world applications. Additionally, TRC is expected to offer more rigorous safety guarantees, as it uses conditional value-at-risk constraints instead of expectation-based constraints. For the TRC method, various training

Table 1

Comparison between different methods on the collision rate (lower is better) of the three tasks. The first subtable presents the results in the training environment where the worker moves forward randomly using a fixed walking posture (i.e., the same posture during training). The second and third subtables aim to investigate the generalization ability. In the second subtable, the worker's velocity is fixed at the maximum. In the third subtable, worker's posture is changed to the standard standing posture.

	Ours	DRL [65] (with collision penalty)	DRL [65] (without collision penalty)	RRT	TRC [53]	MPPI
Training environments (Baseline performance)						
Task (a)	0%	0%	100%	0%	0%	0%
Task (b)	0%	0%	100%	100%	0%	0%
Task (c)	0%	12%	100%	100%	0%	0%
Worker's velocity fixed at maximum						
Task (a)	–	–	–	–	–	–
Task (b)	0%	49%	100%	100%	9%	0%
Task (c)	0%	69%	100%	100%	15%	35%
Worker's posture in standard standing posture						
Task (a)	0%	4%	100%	0%	0%	0%
Task (b)	0%	19%	100%	100%	0%	0%
Task (c)	0%	23%	100%	100%	1%	0%

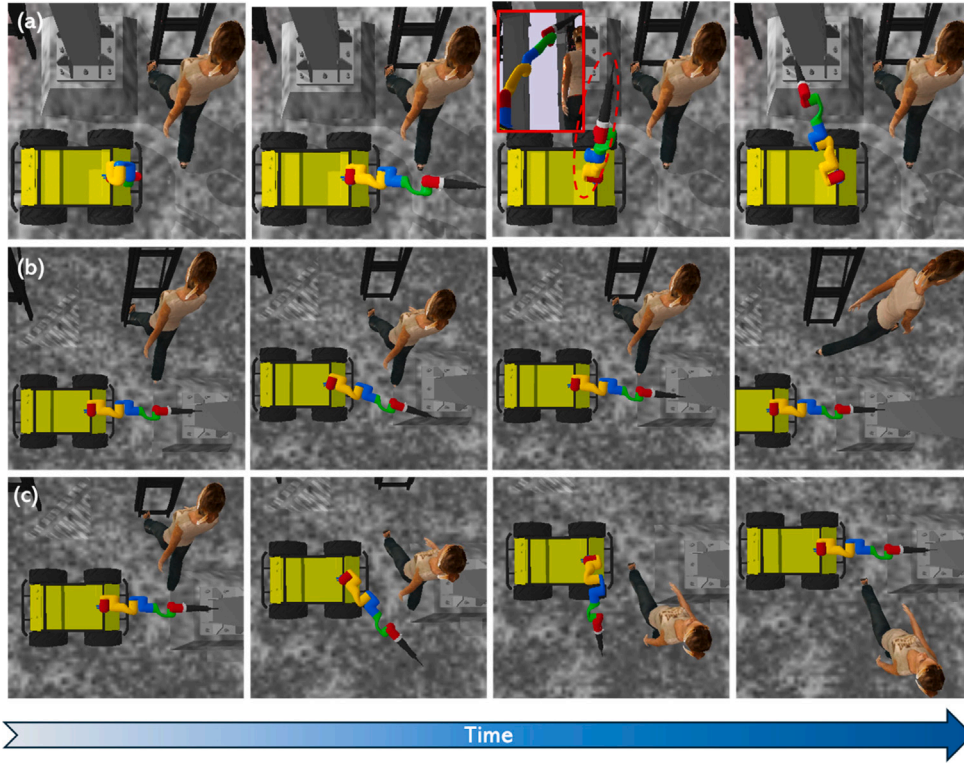


Fig. 9. Results of tasks (a), (b), and (c). The first, second, and third rows are tasks (a), (b), and (c), respectively. Robot movement in the static and dynamic HRC simulation scenarios.

configurations were conducted to identify the optimal cost function design and corresponding coefficients for balancing cost and reward optimization. The tests were conducted using the policy trained with best configurations.

MPPI has been widely used in online trajectory planning such as autonomous driving [70,71]. As a variant of model predictive control, MPPI is particularly well-suited for systems with non-differentiable dynamics, such as those encountered in non-differentiable simulators. We implemented the MPPI framework with built-in safety constraints listed in Eqs. (1)~(4), to enhance its performance. The cost functions for constraints were designed based on reciprocal barrier functions [72], and the reward function used the affine function. The dynamic model used in MPPI is the same as the one presented in Eq. (6), with the low-level controller employing the same PD controller. Additionally, we also utilized the Semi-implicit Euler method to predict the future

states of the robot system and adopted the same assumption on obstacle states. The coefficients of the cost and reward functions, as well as the hyperparameters in MPPI, such as the horizon length and the number of samples, have been determined based on numerous empirical experiments and tuning procedures.

6. Results and discussion

Learning curves of these control methods are shown in Fig. 8. The solid lines are the mean rewards, and the filled areas reflect the standard deviation. Our method used the same neural network policy trained without collision penalty; hence there are only three lines in this figure. As expected, the DRL agent without a collision penalty converged faster. Without the collision penalty, the model converged in approximately 120k steps, while with the collision penalty, it required

around 600k steps and converged to relatively lower rewards (due to the presence of negative rewards). However, the TRC method demonstrated the slowest convergence rate, largely due to its dual-objective optimization approach. It prioritized minimizing the cost function first, driving it to a low threshold, before attempting to maximize the reward within the constraints imposed by the cost function. Moreover, TRC is highly sensitive to the cost function design. Lower costs lead to inadequate collision avoidance, while higher costs significantly slow convergence and degrade overall task performance.

Additionally, the filled area for the DRL without the collision penalty is much narrower after 500k steps, indicating lower variance post-convergence. Consequently, if the proposed control system can avoid collisions when using a neural network policy trained without a collision penalty, the training process will benefit from faster convergence.

The results of collision avoidance are illustrated in the first subtable in Table 1. It is apparent that if the neural network policy did not consider any collision avoidance during training, the robot consistently failed to avoid collisions. However, even when a penalty term was added to the reward function to encourage collision avoidance, collisions still occurred, albeit at a very low rate. Indeed, similar phenomena were observed in previous studies [17,23]. The RRT method performed well in task (a), where the worker remained static, but struggled in dynamic environments due to significant optimization delays, which prevented timely responses to the worker's movement. To ensure collision-free trajectories, RRT relied on robot dynamics to predict motion. However, this dependence limited RRT's ability to compute feasible trajectories in real time, leading to failure in dynamic tasks (b) and (c). The TRC and MPPI methods successfully avoided collisions in tasks (a), (b), and (c).

However, it must be noted that results from the first subtable in Table 1 cannot fully prove the performance of the trained neural network policy in collision avoidance, since the policies may be overfitting to the worker's movement pattern and the environment setting. To illustrate, we conducted two additional experiments for out-of-distribution scenarios.

First, we slightly adjusted the worker's velocity to the maximum velocity, resulting in a collision rate increase to over 50% for the DRL policy trained with collision penalty (see the second subtable in Table 1). The TRC method successfully avoided collisions in tasks (a); however, it exhibited a non-zero collision rate in tasks (b) and (c). This may be because that maximum velocity was not sufficiently sampled during training, preventing the neural network policy from learning to handle these situations fully. As expected, RRT method failed in the dynamic environments. MPPI succeeded in the first two tasks but had a high collision rate in tasks (c). This is mainly because MPPI uses predicted costs and rewards over sampled trajectories to compute the next action. It requires the prediction of future states over a horizon length, which significantly increases the computational cost, especially when using 3D rigid-body dynamic models. This has been reported by other studies [71,73]. For example, in [73], a 3D rigid-body model with tire forces was applied in MPC control for high-speed autonomous vehicles at a control rate of 3 Hz, whereas in this study, MPPI operated at a control rate of 4 Hz. However, with the same dynamic model configurations, our proposed control system succeeded and operated at 100 Hz because we used Eq. (31) instead of optimizing online. Therefore, our control system is more computationally efficient.

Furthermore, altering the worker's posture also led to a significant rise in collision rates for the DRL policy trained with collision penalty, as seen in the third subtable in Table 1. The term "standard standing posture" refers to a way of standing with the feet together and arms by the sides. Realistically, capturing all possible movement patterns is challenging. Therefore, as mentioned earlier, sufficient data is crucial for ensuring the reliability of the neural network policy, yet achieving this remains challenging. Conversely, it can be seen that our proposed method, RRT method, and MPPI method remained unaffected

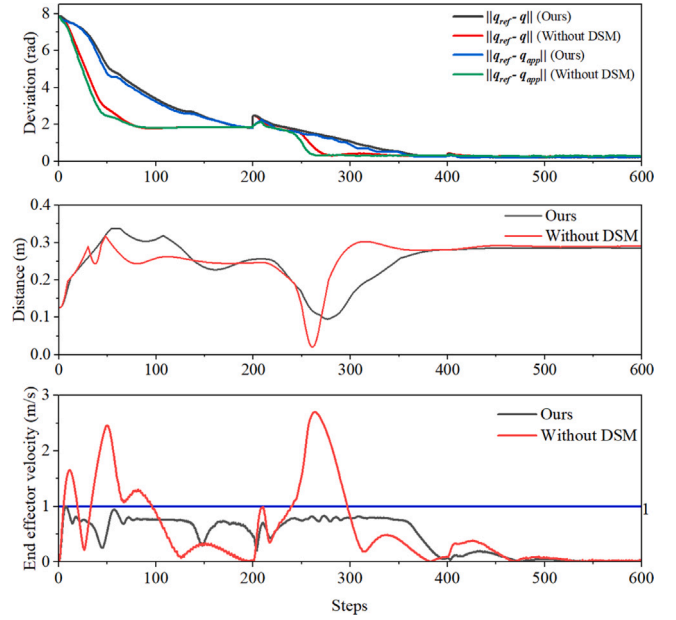


Fig. 10. Results of task (a).

by changes in worker's posture. The TRC method successfully avoided collisions in tasks (a) and (b); however, it was also sensitive to posture changes, leading to a non-zero collision rate and low success rate in task (c).

Nearly all methods excelled in task (a), with TRC and neural network policies incorporating collision penalties also demonstrating robustness to environmental changes. However, unlike RRT and the proposed approach, their performance stemmed partially from exploiting a shortcut: they learned to complete the task by constraining robot movements within the desk's boundaries, effectively ignoring the worker's motion as long as the worker remains outside of the desk. However, this approach can be problematic in environments where the robot operates on a smaller platform, as it limits adaptability to different worker locations.

To conclude, since the proposed method adopts an explainable scheme for collision avoidance and does not require the reward function to include a penalty term for collisions or rely on defined cost function for constraint adherence, it reduces the challenges in reward function design, has faster convergence speed (see Fig. 8), and improves collision avoidance performance (see Table 1).

Next, to evaluate the joint actions closely, we focus on the proposed method and discuss its quantitative and qualitative performance.

6.1. Static environment results

In this environment, the robot will follow a working trajectory generated by the neural network policy to perform construction tasks. During its movement to the target working area, the robot must adjust its original commands from the neural network policy to avoid unsafe actions. The neural network policy generates q_{ref} every 2 s (0.5 Hz), and the reference governor layer rapidly responds to environmental stimuli to ensure safety (100 Hz). The low-level PD controller was operating at 1 kHz. Ideally, the reference governor layer will prevent the robot from violating safety constraints, even if q_{ref} suggests otherwise.

Qualitative results are illustrated in the first row of Fig. 9. The robot initially remained in a state where all joint angles were set to zero. Then, the neural network policy generated a reference trajectory q_{ref} for the robot arm. In the initial stage, the robot was able to navigate with q_{ref} with little influence from the constraint conditions (see the second figure in task (a) in Fig. 9 where the robot slightly moved backwards).

The constraint conditions imposed a repulsion joint vector p_{rep} on the current applied reference joint angles q_{app} to change the navigation direction of the robot arm. The magnitude of the modification was under constraints of δ_{RH} , τ_{limit} , q_{limit} , q_{limit} , and x_{elimit} . The modified trajectory successfully avoided potential collisions with the worker, and always maintained a safe distance, as shown in the third figure in task (a) in Fig. 9. This implies that although the neural network policy did not consider any collision avoidance, safe HRC was naturally achieved by applying safety-constrained vector fields on q_{ref} . Finally, the robot arm followed a new trajectory, bypassed the worker, and navigated to the desired position.

Fig. 10 presents the quantitative results in the static environment scenario. The first subplot shows the difference between q_{ref} , q_{app} , and q . The second subplot shows the minimal distance between the robot and the surface of the spheres. The third subplot shows the velocity of the end effector. The term “Without DSM” refers to a control system that operates without dynamic safety guarantees for the robot. Specifically, the parameter D remains constant across the tasks. The robot will also adjust its direction but will update q_{app} with a constant derivative.

In Fig. 10, with the PD controller, q is nearly equal to q_{app} to execute the commands received by actuators. In this environment, to accomplish task (a), the neural network policy initially generates the first reference joint action q_{ref} for the actuators. This leads to significant deviations, as the robot starts from its initial state at time 0 and needs to move a distance to reach the target positions. Therefore, the deviation at time 0 in this environment represents the distance the robot must travel from its initial state to the target positions. As the robot arm began to move, the deviation and the minimal distance between the robot and the spheres surrounding the worker started to decrease. The end effector velocity increased to the velocity limitation 1 m/s and then decreased due to the constraints. During this phase, the robot needs to slightly adjust its action by maximizing the distance between the robot and the worker and minimizing the distance between the end effector and the target position. As a result, the robot cannot quickly converge directly to q_{ref} . After 2 s, the neural network policy generated the second q_{ref} , causing a brief spike in the deviation. However, since this q_{ref} was already close to the destination, the spike was not significant. The robot continued moving towards the target position, reducing the distance to the target while maintaining a safe distance from the worker. It changed its original trajectory to avoid collisions and eventually found a new feasible path, allowing it to move away from the worker while continuing towards its destination. During this process, the end effector velocity did not exceed the maximum velocity. After 4 s, the neural network policy generated the final q_{ref} . Similarly, because this q_{ref} was even closer to the destination, the changes in the deviation were minimal. Consequently, the robot converged to the new q_{ref} , and the end effector velocity began to decrease.

Note that we did not include all DSM components in Fig. 10. Since our method ensures a safe DSM and the robot stops when the DSM is zero, not all of the DSM components fall below zero in these three tasks. Therefore, we chose to display the DSM components that showed significant differences and posed safety risks without the DSM (see Section 6.4).

6.2. Dynamic environment results

In this scenario, the worker moved towards the robot while the robot focused on its task. The neural network policy, reference governor layer, and PD controller were operated at 0.5 Hz, 100 Hz, and 1 kHz, respectively. Tasks in this environment were designed to evaluate whether the robot's intuitive reactions can ensure safety in a dynamic environment. The safety constraints are expected to push the robot arm away from the worker, while the work requirements continuously generate a reference trajectory to navigate the robot arm back to its

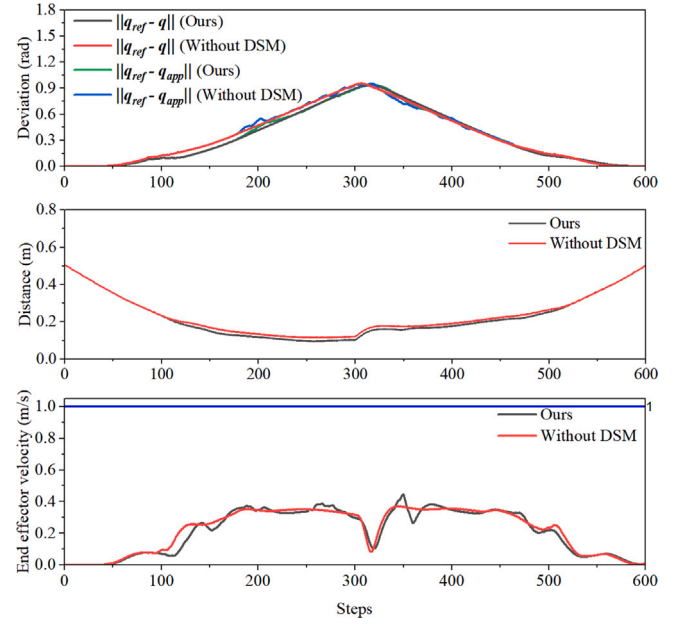


Fig. 11. Results of task (b).

original work area. Ideally, the robot will first move away from the worker and then return to the work area once the safety conditions are satisfied.

The second row in Fig. 9 shows the robot system's performance under a slight disturbance, where the worker first moved towards the robot and then moved far away from the robot. The robot slightly moved away from the worker to maintain a safe distance and restored to the initial state once the worker moved away.

Fig. 11 shows quantitative results. The first subplot shows the difference between q_{ref} , q_{app} , and q . The second subplot shows the minimal distance between the robot and the surface of the spheres. The third subplot shows the velocity of the end effector. At the early stage, the robot remained stationary until the worker reached the safety margin δ_{RH} . After that, the robot started to move away from the worker. When the worker stopped and prepared to return, the velocity of the worker reduced, and the minimal distance between the robot and the surface of the spheres was reached. The end effector's velocity also decreased as the worker began to return. As the worker moved back, the end effector's velocity started to increase again. Finally, the robot returned to its work area and converged to its initial position.

In task (c), the worker might walk across the robot's working trajectory, forcing the robot to continue to maintain a safe distance with the limitations of safety constraints including δ_{RH} , τ_{limit} , q_{limit} , q_{limit} , and x_{elimit} .

Task (c) is more challenging than task (b) due to the larger disturbance. The robot is expected to move a significant distance from its original work area to ensure safety, while the attraction vector p_{att} pushes the robot back to the work area. Once the robot reaches a distance where the repulsion vector p_{rep} reaches zero, it will move back at high speed. As the worker moves across the robot's work area, there will be a point where the worker starts to move away from the robot. After this point, the p_{rep} can be zero, and the robot will have a large dynamic safety margin, leading to a fast return velocity, which can violate the safety constraints. Our approach mitigates this effect.

As shown in Fig. 9, the robot can work properly under significant disturbance. It maintained a safe distance during worker's movement, and resumed its work state once the worker was in a safe state. The quantitative results are shown in Fig. 12. The first subplot shows the difference between q_{ref} , q_{app} , and q . The second subplot shows the minimal distance between the robot and the surface of the spheres. The

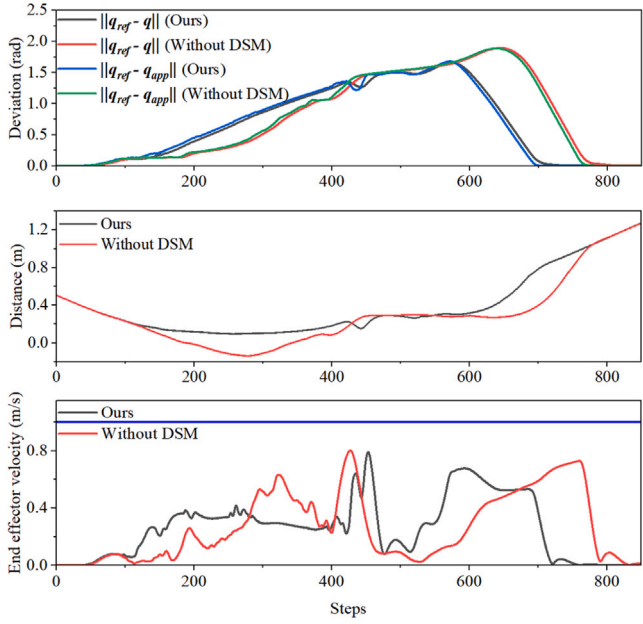


Fig. 12. Results of task (c).

third subplot shows the velocity of the end effector. Similar to task (b), the robot initially remained stationary until the worker reached the safety margin δ_{RH} . After that, the robot arm continued to maintain a safe distance. After 8 s, the robot determined it was safe to move back. Due to the influence of the DSM, the robot did not exceed the maximum end effector velocity. As the robot approached the end effector velocity limit $\dot{x}_{elimit}$, both the DSM D and the velocity began to decrease. Consequently, there was a flat platform around 9 s, indicating that D decreased to a very small value.

6.3. Computation cost

The computational costs were recorded during the experiments. The average computation time for the reference governor layer is 0.00565 s, and the average computation time for the PD controller is 1.59×10^{-6} s. The average inference time of the neural network policy varies depending on the network architecture. In this study, a simple network architecture was utilized, resulting in an average inference time of 0.00147 s. All computations were performed using Python on a laptop-class i7-13620H CPU. Computation can be further accelerated by using C or C++.

6.4. Ablation study

This section aims to reveal the influence of DSM. Several control systems were operated without DSM. “Without DSM” refers to the proposed control system that operates without dynamic safety guarantees. Specifically, the parameter D remains constant across all tasks. The robot will also adjust its direction but will update q_{app} with a constant derivative. We mainly focus on tasks (a) and (c) in this section since the small disturbance in task (b) did not cause significant differences when using DSM (see Fig. 11). In task (b), the control system functions effectively under constraints regardless of DSM, because human-robot interaction only reduces the DSM to a small value rather than below zero (i.e., no safety constraints are violated during this task execution). If the interaction leads to an actual safety constraint violation, the DSM will be activated and set to zero to prevent such violations.

Compared with the proposed method, the method without DSM in Fig. 10 shows a more aggressive movement. For example, its convergence to q_{ref} was faster, allowing the robot to better follow commands

of the neural network policy. However, the minimal distance between the robot and the surface of the spheres was close to zero at around 2.5 s, and the maximum end effector velocity significantly exceeded $\dot{x}_{elimit}$ (i.e., the blue solid line). In addition, our method is expected to achieve the same convergence speed as methods without DSM when the robot operates under safe conditions (see Fig. 11).

In task (c), the deviation is significantly larger compared to task (b), the robot will increase its velocity to return to the working area after the worker moves away. As a result, both methods in Fig. 12 show a decrease in the safety distance. However, the method without DSM in Fig. 12 indicates a collision between the worker and the robot since the distance falls below zero. After the collision, the robot is “pushed away” by the force exerted from the human body. The DSM is necessary to ensure dynamic safety, as the direction modifier can only adjust the direction.

Therefore, it is evident that the DSM ensures a safer HRC by maintaining safety conditions within acceptable limits.

6.5. Real robot test results

The results of the real robot tests are illustrated in Fig. 13. We performed three tasks during the test. The first row shows the results of task (a), while the second and third rows display the results of tasks (b) and (c), respectively.

In task (a), the robot moved to the desired position while maintaining a safe distance from the worker. Without the reference governor, the robot arm would have collided with the worker’s left arm. In tasks (b) and (c), the worker moved towards the robot, causing the robot to automatically move away to maintain safety. Once the worker left the robot’s working trajectory, the robot moved back to its initial position.

The results of the real robot tests with various postures are illustrated in Fig. 14. Over a duration of 150 s, the worker used different postures to approach the robot arm, evaluating whether the robot could ensure safety. As discussed in the previous section, conventional DRL methods are sensitive to diverse postures.

In Fig. 14, the worker first approached the robot with the left hand at $t = 22$ s and $t = 26$ s, and the robot arm successfully moved to a safe position. After the worker retracted the left hand, the robot arm returned to its work area at $t = 36$ s. Then, the worker approached with the right hand at $t = 44$ s, and the robot moved back to its work area when it detected the area was safe at $t = 52$ s. Next, the worker approached the robot arm from the bottom at $t = 57$ s, and the robot arm successfully avoided collisions and returned to its position at $t = 70$ s. Following this, the worker moved the left and right hands from the camera side to the robot arm, and the robot arm avoided potential collisions at $t = 77$ s and $t = 84$ s, respectively. Finally, the worker approached the robot using both hands from two sides (i.e., the left hand on the worker side and the right hand on the camera side), and the robot moved away from the worker at $t = 130$ s and $t = 132$ s. Therefore, this task demonstrates the robustness of our method with respect to worker postures and its effectiveness in avoiding safety risks during construction HRC. Video demonstration: <https://kangk906.github.io/safetycon.github.io/>

7. Conclusions

Safety is a crucial concern in the application of HRC in the construction industry. To address this issue, this paper proposed a biomimetic safety-constrained DRL method. The proposed method theoretically enables safe and reliable HRC in on-site construction, laying the groundwork for a future revolution in construction technology. This method mimics the decision-making process of the central nervous system of vertebrates. A neural network policy functions as the brain, generating reference joint actions and handling construction tasks. A reference governor acts as the spinal cord, operating at high frequency and producing intuitive reactions to ensure safety. The reference governor layer

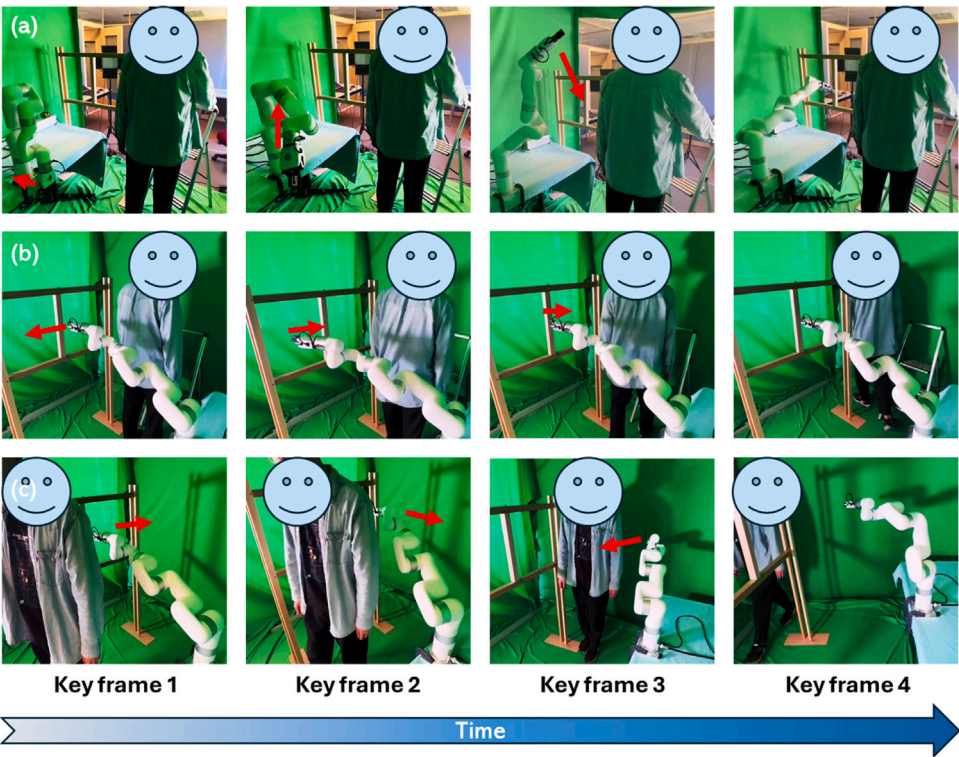


Fig. 13. Results of real robot test on three tasks.

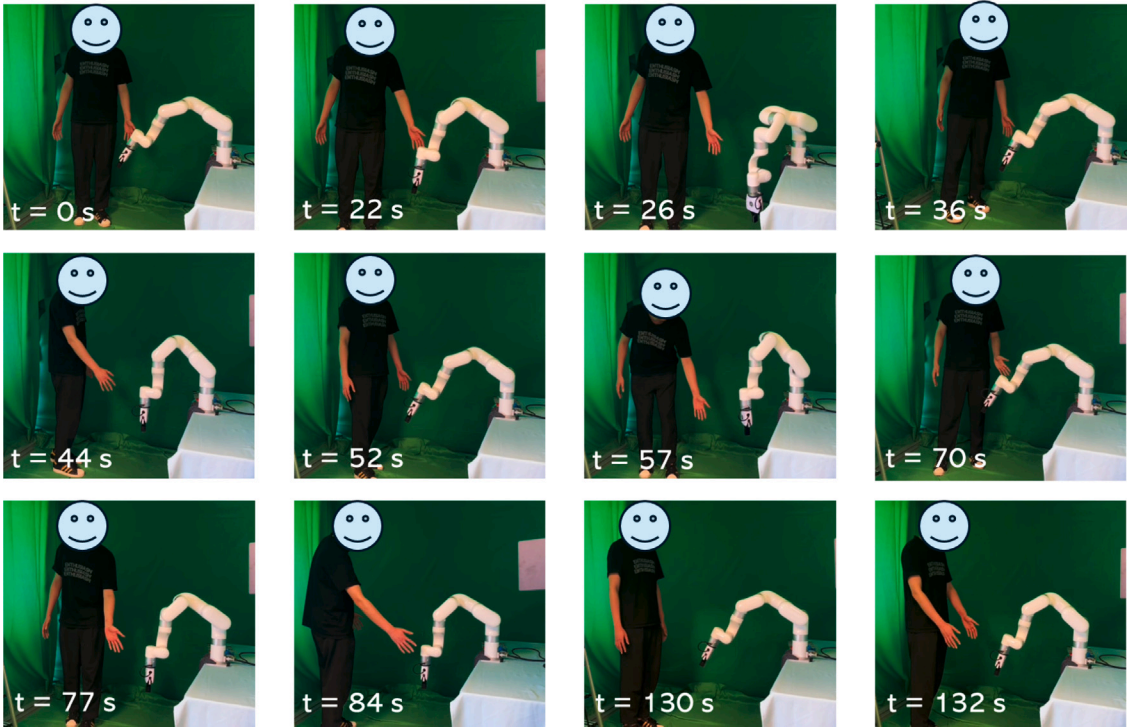


Fig. 14. Results of real robot test under diverse postures.

ensures rapid safety responses, enabling the robot to act autonomously to meet safety constraints without relying on the neural network policy (the brain) for immediate decision-making. We evaluated our method in both static and dynamic environments through simulation and real robot experiments and drew the following conclusions.

- The neural network policy alone cannot provide sufficient safety in HRC. Conventional DRL-based control learns from trial and error, optimizing policies to maximize rewards through numerous iterations. It is challenging to determine the amount of data required for a neural network policy to generate a robust safety policy. Otherwise, the neural network policy is likely to overfit to the training dataset according to our experiments. Additionally, the neural network policy is a black box model and lacks explainability.
- The proposed safety-constrained DRL method not only provides theoretical safety guarantees but also outperforms conventional DRL control in terms of safety across various environments, achieving a 0% collision rate in experiments. In real-world scenarios, this method offers more reliability due to the theoretical safety guarantees it provides. This reliability is particularly important for applications in on-site construction.
- The proposed safety-constrained DRL method addresses the lack of theoretical safety guarantees while preserving the learning capabilities of conventional DRL methods. In this biomimetic control system, the neural network policy functions as the brain, utilizing its powerful approximation ability to approximate the optimal policy for completing construction tasks. The reference governor layer operates at a higher frequency to ensure a rapid response to environmental stimuli, acting as an intuitive safety mechanism. This design allows the robot to update the neural network policy without considering safety constraints, resulting in faster convergence compared to conventional DRL methods (5 times convergence speed in this paper). This advantage is crucial for the application of construction robots in complex environments, where safety-related constraints can significantly increase convergence time.
- The proposed method exhibits adaptability to environmental changes. In task (a), we observed that the robot employing the proposed control system did not halt its actions when a worker is in its planned trajectory. Instead, it successfully identified a new trajectory that circumvented the worker and reached the target place. Conventional DRL methods learn explicit policies represented by a single model distribution. In contrast, the reference governor layer introduces a secondary distribution that encodes safety constraints. This enables adaptive modification of the policy distribution, allowing for the generation of feasible trajectories while adhering to safety requirements. As a result, the proposed approach outperforms traditional construction robots, which rely solely on rule-based interventions to halt operations upon constraint violations.
- Unlike conventional methods, which approximate dynamics using neural network and limit in collision safety, the proposed method explicitly considers robot dynamics, achieving safer behavior in real-world motion with limits on collisions, torques, joint positions, velocities, and end-effector speeds. Given that the actual trajectory $\mathcal{T}_{gt}$ often diverges from the planned trajectory $\mathcal{T}$, we incorporate robot dynamics to adjust $\mathcal{T}$ to ensure it remains steady-state admissible. This is essential in construction, where high-power, heavy-load machinery, especially hydraulic systems, is prone to issues like overshooting and heat buildup. Overlooking control safety such as velocity limits in such environments risks damaging equipment and leading to scenarios where approximated dynamics become unreliable.

In this study, the experiments cover situations where the robot and worker operate in the same zone but carry out tasks independently. In tasks that involve close contact, such as material conveyance, the nominal safety radius for the worker may need to be reduced to ensure that the robot can receive the materials. However, further work is also needed to refine the interaction patterns in such complex HRC scenarios. For instance, determining the nominal safety radius requires careful consideration of task requirements, robot dynamics, and the type of end-effector used. Additionally, the psychological comfort of workers is crucial so robot movements should not appear overly aggressive from the workers' perspective.

However, there are limitations in our method. We assume the worker is static while calculating the potential future state of the robot, which might induce safety concerns. In addition, high relative speeds between the worker and the robot, particularly during opposing movements or interactions involving a worker's limb, can lead to collisions due to the finite maximum speed of the robot. The robot's ability to respond depends on factors such as its power and load—robots with higher power are better equipped to move quickly to avoid collisions, while heavier loads make such evasive actions more difficult. To minimize the affects, one can increase δ_{RH} , run the reference layer at high frequency or use deep learning-based models to predict workers' future trajectories. A reliable trajectory prediction model enables robots to anticipate movements in highly dynamic environments. With DSM, the control system can ensure that the robot halts reference updates if any safety violations could be found within a short future time window.

Another limitation of the current method is that it does not fully account for cases where the robot manipulates large objects that extend its reach, such as long rebar. In such situations, the object can be integrated into the robot's kinematic model as an additional link. Future work will focus on extending our approach to handle such scenarios, ensuring that the extended reach is appropriately considered during trajectory planning and collision avoidance.

Theoretically, the proposed method extends to multi-robot scenarios. Future work will focus on applying this approach in diverse complex on-site construction environments where multiple robots and workers interact. Additionally, the choice of human body model directly impacts the calculation of minimum distances; refining these models for multiple workers could further enhance computational efficiency.

CRediT authorship contribution statement

Kangkang Duan: Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Zhengbo Zou:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors acknowledge the support of the Natural Sciences and Engineering Research Council of Canada (NSERC). Kangkang acknowledges the support of the CSC scholarship from the Chinese Scholarship Council. The experiments were approved by the authors' institutional review board under the protocol number H22-01586.

Appendix A

A.1. Details about DRL training

The basic information of the neural network policy is illustrated in Table 2. Define the size of the observation and action space are n_o and n_a , respectively. The reward function was designed for a reaching task. The learning performance of the neural network policy in construction tasks was demonstrated in [23,24,36–39]. Our proposed method focuses on safety behavior, which can be effectively demonstrated through a simple reaching task where collisions between the worker and the robot are the main challenges. Once the robot reaches the desired area, it can be further trained to install beams, grasp bricks, or perform other tasks. Therefore, we utilized the reaching task to evaluate our method. The reward function r_t is:

$$r_t = -0.05 \|p_{des} - x_e\| \quad (32)$$

where p_{des} and x_e are the desired and end effector positions at time t , respectively. If the robot completes the task, it will receive a reward of 1.0. If the robot collides with the spheres, it will receive a penalty of -1.0. The inputs to the neural network policy have been normalized using their respective maximum values.

TRC method also used the same neural network structure, and Eq. (32) for reward computation. In addition, the cost function was a rectifier function based on the distance between points in R and H .

$$c_t = \left[k_c \min_{i=1, \dots, n; j=1, \dots, k} (\|d_{ij}\| - r_j) \right]_+ \quad (33)$$

Fig. 15 shows the distribution of the desired positions and the worker's positions. It is important to note that the worker can randomly move in each episode. For each step, the worker can move forwards within a range of -0.1 m to 0.3 m in the area. The worker can also move right or left within a range of -0.05 m to 0.05 m. This is important in training since if the movement pattern is fixed to moving forward during training, the robot can intelligently adapt by learning to retract its arm once an episode commences. In this situation, even the worker does not move to the robot, the robot will retract its arm.

Task (a) does not require a new neural network policy because it only includes static obstacles and we use dynamic obstacles for all tasks. We can simply fix the positions of the obstacles during the execution of Task (a). Policy trained in dynamic environments can also succeed in static environments. Additionally, we want the robot to cross the worker's position area during task execution to determine if it has learned to avoid collisions rather than merely over-fitting to the worker's position distribution. Therefore, we specified that the robot must reach the right blue area before it can proceed to the top blue area.

A.2. Details about evaluation

The initial positions and movement patterns (e.g., direction and step size) were randomized for each of the 100 tests. During evaluation, except for task (a) where the worker remains static, the worker moved randomly. Specifically, at each step, the worker could move forward within a range of 0 to 0.3 m and left or right within a range of 0 to 0.05 m. For all tasks, the worker's initial position was randomized in a subarea of the red area in Fig. 15 so that the initial position also ensures that the worker would cross paths with or approach the robot's planned trajectory. For task (a), the desired position for the robot arm was also randomized in the blue area for each test. The worker's posture remained fixed during each test, meaning no shoulder or hand movements were allowed. This setup was designed to validate the control system's performance in each task.

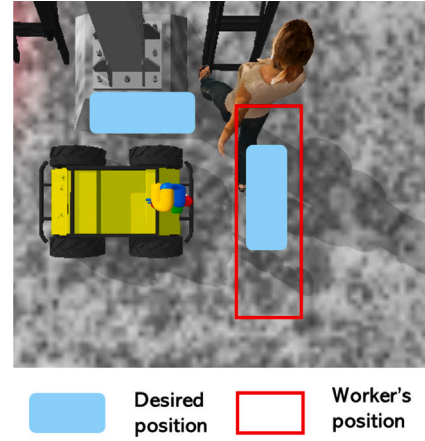


Fig. 15. Distribution of desired positions and worker's positions in DRL training.

Table 2

Configurations of the neural network policy.

Configurations of the robot	
Degree of freedom	6
Maximum velocity	1 m/s
Maximum joint velocity	180°/s
Joint 1	±360°
Joint 2	-118°~120°
Joint 3	-225°~11°
Joint 4	±360°
Joint 5	-97°~180°
Joint 6	±360°
Weight	12 kg
Reach	700 mm
Payload	5 kg
Configurations of the neural network policy	
Size	$[n_o, 512, 512, n_a]$
Activation function	Hyperbolic
Max steps	6,000,000
Policy learning rate	0.001
Value function learning rate	0.001
ζ	0.10
γ	0.99
ξ	0.97
Other configurations	
Nominal safety radius	0.2 m~0.35 m
δ_{RH}	0.4 m
ρ_{RH}	0.03 m
δ_q	10°
ρ_q	3°
η	0.01
k_τ	25
k_q	100
$k_{\dot{q}}$	75
$k_{\ddot{q}}$	75
$k_{\ddot{x}_e}$	100
k_{RH}	15
δ_{th}	15
k_c	-15

A.3. Details about real robot test

The Intel RealSense D435i depth camera was used in the experiments. Instead of relying on a 3D pose estimation model, an off-the-shelf 2D pose estimation model was utilized, in combination with depth images, to achieve accurate 3D position estimation. The model could predict 2D joint coordinates on RGB images, which are then converted to 3D coordinates using the camera's intrinsic parameters and corresponding depth values. Due to depth noise, each joint's depth at coordinates (x, y) was refined by taking the median value from a

larger surrounding area, $\{(x, y) \in \mathbb{R}^2 \mid x - \Delta x \leq x \leq x + \Delta x \text{ and } y - \Delta y \leq y \leq y + \Delta y\}$, to improve accuracy. Both Δx and Δy were set to 5 pixels. Results show that this method is computational efficiency and effective. Interpolation was used to generate a series of sphere locations that enclose the human body. To obtain the position of an object, an AprilTag was used, and a larger surrounding area was also employed to reduce noise.

The camera and robot arm were fixed in position in the scenario. The relative position between the camera and robot base was calibrated using a checkerboard. After calibration, human pose data and robot link information were transformed into the robot base frame. In this study, only one camera was used for demonstration. For real applications, one can use multiple cameras, IMU, radio-frequency identification location tracking, or ultra-wideband positioning techniques to reduce the influence of occlusion.

Data availability

Data generated or analyzed during the study are available from the corresponding author by request. Video demonstration: <https://kangk906.github.io/safetycon.github.io/>.

References

- [1] B.U. Ayhan, O.B. Tokdemir, Safety assessment in megaprojects using artificial intelligence, *Saf. Sci.* 118 (2019) 273–287, <http://dx.doi.org/10.1016/j.ssci.2019.05.027>.
- [2] K. Liu, Y. Liu, Y. Kou, Study on construction safety management in megaprojects from the perspective of resilient governance, *Saf. Sci.* 173 (2024) 106442, <http://dx.doi.org/10.1016/j.ssci.2024.106442>.
- [3] A. Giallanza, G. La Scalia, R. Micale, C.M. La Fata, Occupational health and safety issues in human-robot collaboration: State of the art and open challenges, *Saf. Sci.* 169 (2024) 106313, <http://dx.doi.org/10.1016/j.ssci.2023.106313>.
- [4] M. Rüßmann, M. Lorenz, P. Gerbert, M. Waldner, J. Justus, P. Engel, M. Harnisch, *Industry 4.0: The Future of Productivity and Growth in Manufacturing Industries*, Technical Report, Boston Consulting Group, 2015.
- [5] Y. Sun, I. Jeelani, M. Gheisari, Safe human-robot collaboration in construction: a conceptual perspective, *J. Saf. Res.* 86 (2023) 39–51, <http://dx.doi.org/10.1016/j.jsr.2023.06.006>.
- [6] American national standard for industrial robots and robot systems - safety requirements, 2012, URL <https://webstore.ansi.org/standards/ria/ansiriar15062012>.
- [7] C.S. Franklin, E.G. Dominguez, J.D. Fryman, M.L. Lewandowski, Collaborative robotics: New era of human-robot cooperation in the workplace, *J. Saf. Res.* 74 (2020) 153–160, <http://dx.doi.org/10.1016/j.jsr.2020.06.013>.
- [8] X. Zhang, M. Skitmore, Y. Peng, Exploring the challenges to industrialized residential building in China, *Habitat Int.* 41 (2014) 176–184, <http://dx.doi.org/10.1016/j.habitatint.2013.08.008>.
- [9] F. Barbosa, J. Woetzel, J. Mischke, M.J. Ribeiro, M. Sridhar, M. Parsons, S. Brown, Reinventing Construction Through a Productivity Revolution, Technical Report, McKinsey Global Institute, 2017, URL <https://www.mckinsey.com/capabilities/operations/our-insights/reinventing-construction-through-a-productivity-revolution>.
- [10] Technical Report – Industrial Robots and Robot Systems – Safety Requirements – Collaborative Robots, Technical Report, Robotics Industry Association (RIA), 2016, URL <https://www.automate.org/store/products/ria-tr15-606-2016-collaborative-robots-pdf-download>.
- [11] ISO/TS 15066: Robots and robotic devices - collaborative robots, 2016, URL <https://www.iso.org/standard/62996.html>.
- [12] M.F. Milazzo, G. Ancione, V. Spasojević Brkić, Safety in crane operations: An overview on crane-related accidents, in: 6th International Symposium on Industrial Engineering, 2015, pp. 36–39, <http://dx.doi.org/10.13140/RG.2.1.4738.5046>.
- [13] B.S. Dhillon, *Robot Reliability and Safety*, Springer, 1991, <http://dx.doi.org/10.1007/978-1-4612-3148-6>.
- [14] S. You, J.H. Kim, S. Lee, V. Kamat, L.P. Robert Jr., Enhancing perceived safety in human-robot collaborative construction using immersive virtual environments, *Autom. Constr.* 96 (2018) 161–170, <http://dx.doi.org/10.1016/j.autcon.2018.09.008>.
- [15] C.J. Liang, K.M. Lundeen, W. McGee, C.C. Menassa, S. Lee, V.R. Kamat, A vision-based marker-less pose estimation system for articulated construction robots, *Autom. Constr.* 104 (2019) 80–94, <http://dx.doi.org/10.1016/j.autcon.2019.04.004>.
- [16] H. Wu, H. Li, H.L. Chi, Z. Peng, S. Chang, Y. Wu, Thermal image-based hand gesture recognition for worker-robot collaboration in the construction industry: A feasible study, *Adv. Eng. Inform.* 56 (2023) 101939, <http://dx.doi.org/10.1016/j.aei.2023.101939>.
- [17] J. Cai, A. Du, X. Liang, S. Li, Prediction-based path planning for safe and efficient human-robot collaboration in construction via deep reinforcement learning, *J. Comput. Civ. Eng.* 37 (1) (2023) 04022046, [http://dx.doi.org/10.1061/\(ASCE\)CP.1943-5487.0001056](http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0001056).
- [18] S.M. Langari, F. Vahdatikhaki, A. Hammad, Improving the performance of RRT path planning of excavators by embedding heuristic rules, *Adv. Eng. Inform.* 62 (2024) 102724, <http://dx.doi.org/10.1016/j.aei.2024.102724>.
- [19] T. Zhou, Q. Zhu, Y. Ye, J. Du, Humanlike inverse kinematics for improved spatial awareness in construction robot teleoperation: Design and experiment, *J. Constr. Eng. Manag.* 149 (7) (2023) 04023044, <http://dx.doi.org/10.1061/JCEMD4.COENG-13350>.
- [20] Z. Wang, C.R. Garrett, L.P. Kaelbling, T. Lozano-Pérez, Learning compositional models of robot skills for task and motion planning, *Int. J. Robot. Res.* 40 (6–7) (2021) 866–894, <http://dx.doi.org/10.1177/02783649211004615>.
- [21] W. Wan, Y. Wang, Z. Erickson, D. Held, DiffTOP: Differentiable trajectory optimization for deep reinforcement and imitation learning, in: The Thirty-Eighth Annual Conference on Neural Information Processing Systems, NeurIPS 2024, 2024, <https://arxiv.org/abs/2402.05421>.
- [22] N.A. Hansen, H. Su, X. Wang, Temporal difference learning for model predictive control, in: International Conference on Machine Learning, 2022, pp. 8387–8406, <https://arxiv.org/abs/2203.04955>.
- [23] K. Duan, Z. Zou, Enhancing construction robot collaboration via multiagent reinforcement learning, *J. Intell. Constr.* (2025) <http://dx.doi.org/10.26599/JIC.2025.9180089>.
- [24] K. Duan, C.W.K. Suen, Z. Zou, Robot morphology evolution for automated HVAC system inspections using graph heuristic search and reinforcement learning, *Autom. Constr.* 153 (2023) 104956, <http://dx.doi.org/10.1016/j.autcon.2023.104956>.
- [25] Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, Sergey Levine, Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning, in: IEEE International Conference on Robotics and Automation, 2018, pp. 7559–7566, <http://dx.doi.org/10.1109/ICRA.2018.8463189>.
- [26] W.E. Kaufmann, K.I. Andreasson, P.C. Isakson, P.F. Worley, Cyclooxygenases and the central nervous system, *Prostaglandins* 54 (3) (1997) 601–624, [http://dx.doi.org/10.1016/S0090-6980\(97\)00128-7](http://dx.doi.org/10.1016/S0090-6980(97)00128-7).
- [27] Kenneth Saladin, *Anatomy and Physiology: The Unity of Form and Function*, McGraw-Hill, New York, ISBN: 1265328625, 2015, pp. 496–497.
- [28] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, P. Abbeel, Learning universal policies via text-guided video generation, in: Advances in Neural Information Processing Systems, vol. 36, 2024.
- [29] E. Garone, S. Di Cairano, I. Kolmanovsky, Reference and command governors for systems with constraints: A survey on theory and applications, *Automatica* 75 (2017) 306–328, <http://dx.doi.org/10.1016/j.automatica.2016.08.013>.
- [30] M. Ambrosio, A. Cottruelo, E. Garone, A saturation-aware trajectory-based explicit reference governor for a robotic arm, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, 2022, pp. 13641–13646, <http://dx.doi.org/10.1109/IROS47612.2022.9981483>.
- [31] K. Merckaert, B. Convens, C.J. Wu, A. Roncone, M.M. Nicotra, B. Vanderborght, Real-time motion control of robotic manipulators for safe human-robot coexistence, *Robot. Comput.-Integr. Manuf.* 73 (2022) 102223, <http://dx.doi.org/10.1016/j.rcim.2021.102223>.
- [32] B. Belousov, B. Wibrane, J. Schneider, T. Schneider, G. Chalvatzaki, J. Peters, O. Tessmann, Robotic architectural assembly with tactile skills: Simulation and optimization, *Autom. Constr.* 133 (2022) 104006, <http://dx.doi.org/10.1016/j.autcon.2021.104006>.
- [33] A.A. Apolinarska, M. Pacher, H. Li, N. Cote, R. Pastrana, F. Gramazio, M. Kohler, Robotic assembly of timber joints using reinforcement learning, *Autom. Constr.* 125 (2021) 103569, <http://dx.doi.org/10.1016/j.autcon.2021.103569>.
- [34] D. Lee, M. Kim, Autonomous construction hoist system based on deep reinforcement learning in high-rise building construction, *Autom. Constr.* 128 (2021) <http://dx.doi.org/10.1016/j.autcon.2021.103737>.
- [35] S. Cho, S. Han, Reinforcement learning-based simulation and automation for tower crane 3D lift planning, *Autom. Constr.* 144 (2022) <http://dx.doi.org/10.1016/j.autcon.2022.104620>.
- [36] F.A. Azad, S.A. Rad, M. Arashpour, Back-stepping control of delta parallel robots with smart dynamic model selection for construction applications, *Autom. Constr.* 137 (2022) <http://dx.doi.org/10.1016/j.autcon.2022.104211>.
- [37] C. Liang, V.R. Kamat, C.C. Menassa, Teaching robots to perform quasi-repetitive construction tasks through human demonstration, *Autom. Constr.* 120 (2020) 103370, <http://dx.doi.org/10.1016/j.autcon.2020.103370>.
- [38] L. Huang, Z. Zhu, Z. Zou, To imitate or not to imitate: Boosting reinforcement learning-based construction robotic control for long-horizon tasks using virtual demonstrations, *Autom. Constr.* 146 (2023) 104691, <http://dx.doi.org/10.1016/j.autcon.2022.104691>.
- [39] K. Duan, Z. Zou, T.Y. Yang, Training of construction robots using imitation learning and environmental rewards, *Comput.-Aided Civ. Infrastruct. Eng.* (2024) <http://dx.doi.org/10.1111/mice.13394>.

- [40] D. Kim, A. Goyal, A. Newell, S. Lee, J. Deng, V.R. Kamat, Semantic relation detection between construction entities to support safe human-robot collaboration in construction, in: ASCE International Conference on Computing in Civil Engineering, 2019, pp. 265–272, <http://dx.doi.org/10.1061/9780784482438.034>.
- [41] X. Zhou, P.-C. Liao, Weighing votes in human-machine collaboration for hazard recognition: Inferring a hazard-based perceptual threshold and decision confidence from electroencephalogram wavelets, *J. Constr. Eng. Manag.* 149 (9) (2023) 13351, <http://dx.doi.org/10.1061/JCEMD4.COENG-13351>.
- [42] J. Cai, A. Du, S. Li, Prediction-enabled collision risk estimation for safe human-robot collaboration on unstructured and dynamic construction sites, in: *Computing in Civil Engineering*, 2021, pp. 34–41, <http://dx.doi.org/10.1061/9780784483893.005>.
- [43] Y. Nassar, G. Albeaino, I. Jeelani, M. Gheisari, R.R. Issa, Human-robot collaboration levels in construction: Focusing on individuals' cognitive workload, in: *Construction Research Congress*, 2024, pp. 639–648, <http://dx.doi.org/10.1061/9780784485262.065>.
- [44] S. Shayesteh, H. Jebelli, Feasibility of virtual avatar simulator for human-robot collaboration training in construction, in: *Computing in Civil Engineering*, 2021, pp. 1417–1424, <http://dx.doi.org/10.1061/9780784483893.173>.
- [45] A. Wachi, X. Shen, Y. Sui, A survey of constraint formulations in safe reinforcement learning, 2024, [arXiv:2402.02025](https://arxiv.org/abs/2402.02025).
- [46] G. Dulac-Arnold, D. Mankowitz, T. Hester, Challenges of real-world reinforcement learning, 2019, <http://dx.doi.org/10.1007/s10994-021-05961-4>, [arXiv preprint arXiv:1904.12901](https://arxiv.org/abs/1904.12901).
- [47] N. Xiong, Y. Du, L. Huang, Provably safe reinforcement learning with step-wise violation constraints, *Adv. Neural Inf. Process. Syst.* 36 (2024) URL <https://arxiv.org/pdf/2302.06064>.
- [48] J. Achiam, D. Held, A. Tamar, P. Abbeel, Constrained policy optimization, in: *International Conference on Machine Learning*, 2017, pp. 22–31.
- [49] L. Yang, J. Ji, J. Dai, Y. Zhang, P. Li, G. Pan, Cup: A conservative update policy algorithm for safe reinforcement learning, 2022, [arXiv:2202.07565](https://arxiv.org/abs/2202.07565).
- [50] T. Moldovan, P. Abbeel, Risk aversion in Markov decision processes via near optimal chernoff bounds, in: *Advances in Neural Information Processing Systems*, vol. 25, 2012.
- [51] H. Bharadhwaj, A. Kumar, N. Rhinehart, S. Levine, F. Shkurti, A. Garg, Conservative safety critics for exploration, 2020, [arXiv:2010.14497](https://arxiv.org/abs/2010.14497).
- [52] Q. Yang, T.D. Simão, S.H. Tindemans, M.T. Spaan, Safety-constrained reinforcement learning with a distributional safety critic, *Mach. Learn.* 112 (3) (2023) 859–887, <http://dx.doi.org/10.1007/s10994-022-06187-8>.
- [53] D. Kim, S. Oh, TRC: Trust region conditional value at risk for safe reinforcement learning, *IEEE Robot. Autom. Lett.* 7 (2) (2022) 2621–2628, <http://dx.doi.org/10.1109/LRA.2022.3141829>.
- [54] Q. Yang, T.D. Simão, S.H. Tindemans, M.T. Spaan, WCSAC: Worst-case soft actor critic for safety-constrained reinforcement learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 10639–10646, <http://dx.doi.org/10.1609/aaai.v35i12.17272>, 12.
- [55] S. Gu, L. Yang, Y. Du, G. Chen, F. Walter, J. Wang, A. Knoll, A review of safe reinforcement learning: Methods, theories and applications, *IEEE Trans. Pattern Anal. Mach. Intell.* (2024) <http://dx.doi.org/10.1109/TPAMI.2024.3457538>.
- [56] I. Usmanova, Y. As, M. Kamgarpour, A. Krause, Log barriers for safe black-box optimization with application to safe reinforcement learning, *J. Mach. Learn. Res.* 25 (171) (2024) 1–54.
- [57] Y. As, B. Sukhija, A. Krause, Safe exploration using Bayesian world models and log-barrier optimization, 2024, [arXiv:2405.05890](https://arxiv.org/abs/2405.05890).
- [58] R. Liu, F. Nageotte, P. Zanne, M. de Mathelin, B. Dresp-Langley, Deep reinforcement learning for the control of robotic manipulation: a focussed mini-review, *Robotics* 10 (1) (2021) 22, <http://dx.doi.org/10.3390/robotics10010022>.
- [59] J.F. Fisac, A.K. Akametalu, M.N. Zeilinger, S. Kaynama, J. Gillula, C.J. Tomlin, A general safety framework for learning-based control in uncertain robotic systems, *IEEE Trans. Autom. Control* 64 (7) (2018) 2737–2752, <http://dx.doi.org/10.1109/TAC.2018.2876389>.
- [60] E.G. Gilbert, K.T. Tan, Linear systems with state and control constraints: The theory and application of maximal output admissible sets, *IEEE Trans. Autom. Control* 36 (9) (1991) 1008–1020, <http://dx.doi.org/10.1109/9.83532>.
- [61] S. Sridhar, A. Oulasvirta, C. Theobalt, Interactive markerless articulated hand motion tracking using RGB and depth data, in: *IEEE International Conference on Computer Vision*, 2013, pp. 2456–2463, <http://dx.doi.org/10.1109/ICCV.2013.305>.
- [62] A. Lakshminarayanan, S. Sharma, B. Ravindran, Dynamic action repetition for deep reinforcement learning, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, 2017, <http://dx.doi.org/10.1609/aaai.v31i1.10918>, 1.
- [63] B. Siciliano, L. Sciacovico, L. Villani, G. Oriolo, *Robotics: Modeling, Planning, and Control*, Springer, London, 2010, <http://dx.doi.org/10.1007/978-1-84628-642-1>.
- [64] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, High-dimensional continuous control using generalized advantage estimation, 2018, [arXiv:1506.02438](https://arxiv.org/abs/1506.02438).
- [65] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, [arXiv:1707.06347](https://arxiv.org/abs/1707.06347).
- [66] M.M. Nicotra, E. Garone, The explicit reference governor: A general framework for the closed-form control of constrained nonlinear systems, *IEEE Control Syst. Mag.* 38 (4) (2018) 89–107, <http://dx.doi.org/10.1109/MCS.2018.2830081>.
- [67] Erwin Coumans, Yunfei Bai, Pybullet, a python module for physics simulation for games, robotics and machine learning, 2021.
- [68] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, Yaser Sheikh, OpenPose: Realtime multi-person 2D pose estimation using part affinity fields, in: *IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1302–1310, <http://dx.doi.org/10.1109/CVPR.2017.143>.
- [69] K. Daniilidis, Hand-eye calibration using dual quaternions, *Int. J. Robot. Res.* 18 (3) (1999) 286–298, <http://dx.doi.org/10.1177/02783649922066213>.
- [70] K. Lowrey, A. Rajeswaran, S. Kakade, E. Todorov, I. Mordatch, Plan online, learn offline: Efficient learning and exploration via model-based control, in: *International Conference on Learning Representations*, 2019, <http://dx.doi.org/10.48550/arXiv.1811.01848>.
- [71] T. Han, A. Liu, A. Li, A. Spitzer, G. Shi, B. Boots, Model predictive control for aggressive driving over uneven terrain, in: *Robotics: Science and Systems*, 2024, <http://dx.doi.org/10.48550/arXiv.2311.12284>.
- [72] A.D. Ames, J.W. Grizzle, P. Tabuada, Control barrier function based quadratic programs with application to adaptive cruise control, in: *53rd IEEE Conference on Decision and Control*, 2014, pp. 6271–6278, <http://dx.doi.org/10.1109/CDC.2014.7040372>.
- [73] S. Yu, C. Shen, T. Earsal, Nonlinear model predictive planning and control for high-speed autonomous vehicles on 3D terrains, in: *IFAC-PapersOnLine*, vol. 54, 2021, pp. 412–417, <http://dx.doi.org/10.1016/j.ifacol.2021.11.208>, 20.